# Online Resource 1 — Supplementary material for: Deep Green AI: Energy Efficiency of Deep Learning across Language–Framework Ecosystems

Leonardo Pampaloni[1*], Marco Pagliocca[1*], Enrico Vicario[1] and Roberto Verdecchia[1]

[1]University of Florence, Florence, Italy.

*Corresponding author(s). E-mail(s): leonardo.pampaloni1@edu.unifi.it; marco.pagliocca@edu.unifi.it;
Contributing authors: enrico.vicario@unifi.it; roberto.verdecchia@unifi.it;

This document is Online Resource 1 of the article *Deep Green AI: Energy Efficiency of Deep Learning across Language–Framework Ecosystems*. It carries the material the article condenses or leaves out: the related-work comparison table, the full experiment specification and execution detail, supplementary analyses for RQ1, RQ3 and RQ4 and for the accelerator-saturation cell, an illustrative industrial scenario, the complete catalogue of defect classes, and the extended threats to validity. Section, table and figure numbers without a letter (Section 6.4, Table 3) refer to the article; lettered ones (Section D.3, Table D.2) refer to this document, and "this paper" in the text below means the article. Every quantity is generated by the same analysis pipeline as the article's and injected at build time.

## A Related work: the comparison table

Table A.1 places this study against the work closest to it on the two axes that matter here: how many language ecosystems are compared, and how energy is measured. Section 3 discusses the studies that deserve more than a row.

**Table A.1**: The closest prior work, on the two axes this study varies. "Stacks" counts distinct language–framework combinations compared under one protocol.

| Study | Stacks | Hardware | Energy instrument | Replication |
|---|---|---|---|---|
| Li et al. (2016) | 4 frameworks, Python only | CPU + GPU | external meter | repeated |
| Pereira et al. (2021) | 27 languages, no DL | CPU | RAPL | 10 runs |
| Gordillo et al. (2024) | 8 languages, no DL | CPU | hardware meter | repeated |
| Georgiou et al. (2022) | 2 frameworks, Python only | CPU + GPU | RAPL + NVML | 30 runs |
| Alizadeh and Castor (2024) | 4 runtimes, Python only | CPU + GPU | software estimator | repeated |
| Li et al. (2024) | 5 bindings, 2 frameworks | CPU + GPU | none (time, memory, accuracy) | repeated |
| Jay et al. (2023) | software meters, not stacks | CPU + GPU | meters against a wattmeter | repeated |
| Marini et al. (2025) | 5 languages | CPU + GPU | software estimator | single run |
| Chattaraj et al. (2025) | 2 languages, no DL | CPU | software estimator | repeated |
| This study | 7 stacks, 5 languages | CPU + GPU | NVML + RAPL counters, estimator alongside | 5 runs |

# B Research methodology, in full

This section gives the parts of Section 4 that the main text condenses: the precision policy as implemented, the specification clause by clause, the history of its conformance checks, the residual divergences, and the detail of the measurement procedure.

## B.1 The precision policy as implemented

All 7 ecosystems run under one precision policy, and it is set rather than inherited: fp32 storage and arithmetic, with TF32 permitted on the accelerator's tensor cores for convolution and for matrix multiplication. For convolution that is what cuDNN does on this hardware unless it is told otherwise, so it is the path a practitioner measures without choosing it; for matrix multiplication it is not, since every binding leaves that path at fp32 by default, and we state both halves rather than set one and inherit the other. A single variable carries the policy — `DEEPGREEN_TF32`, applied through the frameworks for the stacks that import our harness and through `NVIDIA_TF32_OVERRIDE` for the four processes that never do — and every run's manifest records the flags that resulted. Setting it to `0` denies TF32 to all 7. Mixed precision is a separate axis and is enabled in no stack here.

The policy belongs to the specification of Section 4.1 rather than to each ecosystem, and it is there because of what happened when it was not. The superseded campaign pinned TF32 *off* in the shared harness; the pin reached one stack of 7 and no other,

and that single difference was worth more than any difference between bindings this study measures. Section D.4 reports it as an ablation.

## B.2 The specification, clause by clause

A fixed-budget comparison presumes that the budget is the same. In practice each ecosystem is written by a different person against a different API, and the places where they can silently diverge are numerous, individually reasonable and invisible in the energy output: a default learning rate that differs between two framework APIs, a data loader whose worker count follows the machine rather than the protocol, an evaluation loop written one image at a time because the API made that natural, an input tensor left at its native resolution in one stack and resized in another.

We therefore wrote the protocol down before running anything, as a specification with six parts, and made conformance machine-checkable. A seventh part, added before the cell it governs was run, declares the one contrast experiment outside the campaign.

- **S1 — Model.** One canonical definition per architecture. `scripts/export_torchscript_models.py` exports one TorchScript module per (architecture, dataset) pair, and `models/MANIFEST.json` records that module's parameter count and a hash of the parameters in name order. The 3 ecosystems that share the pinned LibTorch build (Python/PyTorch, C++, Rust) load that module; the remaining stacks build the same architecture in their own framework, from the same initialiser and the same batch-normalisation convention, because a framework's defaults for either are not a property of the ecosystem a reader is asking about. The manifest's parameter count is carried into every run as `DEEPGREEN_EXPECTED_PARAMS`, and the three Python stacks, R and Java assert it at startup and refuse to train if it does not match. C++ receives it and does not check it, and neither do the Rust binaries of the $32 \times 32$ campaign; the Rust binaries written for the saturation cell of S7 do. We found this after the campaign and the saturation cell had run, and we report it rather than patch a harness that has already produced the data. C++ and Rust load the same exported module file as PyTorch, which does assert it, so what they lack is the startup assertion, not a verified architecture. Counting parameters is the weak form of the check — two networks can agree on a total and differ layer by layer — so `scripts/verify_architecture_parity.py` builds each stack's model for every cell and compares the sorted multiset of parameter tensor shapes, which is comparable across languages where names and orderings are not: all 7 stacks agree, shape for shape, on all six cells. R links a LibTorch of its own and its binding cannot switch a script module between training and evaluation mode, so it builds the architecture rather than loading the module; it belongs to the LibTorch lineage but not to the control group. In the audited campaign this clause asserted the parameter check and nothing implemented it; Section I records what that cost, and the C++ and Rust gap above is where it is still not implemented.

- **S2 — Optimisation.** One optimiser, one learning rate, one batch size, one epoch count, one loss and one seeding policy. The learning rate and the batch size are checked against the source of every file the campaign executes — universally, rather than by finding one file that matches; the rest is asserted by the specification and was checked by reading.
- **S3 — Pipeline.** One input specification: $32 \times 32 \times 3$ images in raw $[0, 1]$ with no per-stack normalisation, resized once offline so that no framework's resampler enters the comparison; a bounded and equal number of loader workers; batched evaluation at the training batch size; and the whole 10 000-image test split evaluated in every stack, with no final partial batch dropped and no step count rounded down. Every run writes a fingerprint of the split it decoded — mean, standard deviation and range over its 30 720 000 pixel values — so the pipelines are demonstrated to agree rather than asserted to.
- **S4 — Backend.** One LibTorch build across the 3 stacks that share it, and one precision policy for all 7: fp32 storage, TF32 allowed, stated to the CUDA libraries campaign-wide. The accelerator is asserted against the built artefact and not only against the source: `readelf -d` on every Rust binary we build must show a CUDA library among its `NEEDED` entries, and the harness refuses to start on a CPU device rather than warning about it.
- **S5 — Measurement.** One measurement bridge, one sampling interval, one tracking mode, one set of counters, one unit, and one per-epoch record — training loss, test loss and test accuracy — persisted by every stack alongside energy, checked against the campaign's own metric files rather than against the source that writes them. Training accuracy is recorded by the 2 stacks whose training loops already compute it, and is read by no analysis in this paper; the schema names those stacks rather than implying 7.
- **S6 — Replication.** Each configuration is executed 5 times as independent processes with distinct seeds, interleaved rather than run back to back. The seed is threaded from the run contract into each stack's data order and its initialisation, and two of the three ways a stack can ignore the seed it is handed are now checked against the stacks' source rather than against the campaign planner alone.
- **S7 — Accelerator-saturation cell.** One declared contrast cell, run apart from the campaign and never pooled with it. S1, S4, S5 and S6 hold as written — the same 7 stacks, the same two architectures exported once as ten-class modules, the same instruments and 5 interleaved repetitions. S2 keeps Adam at $1 \times 10^{-4}$ for 30 epochs and changes the batch size, to 32 in training and evaluation alike. S3 changes the data: Imagenette, ten ImageNet classes, resized once offline to $224 \times 224$ (shorter side, then a centre crop). Section C.3 gives the reasons for each deviation.

The specification is enforced by 110 automated checks that read the source of every ecosystem, the configuration of every tracker, the built binaries and the campaign's own metric files, and fail the build when a stack drifts out of conformance. This matters more than it may appear. A specification that lives only in a paper is a description of intent; the same specification expressed as an executable check is a property of the

artefact, and it fails at the moment of the divergence rather than at peer review. 12 of the defects catalogued in Section I were introduced by us while building or repairing this study; most were caught by a mechanism rather than by inspection, and two were not, which is the subject of what follows. Several are now enforced by checks under S1, S3 and S5, so the same divergence cannot recur.

## B.3 What the checks caught, and what they could not

Of the 110 checks, 110 pass and 0 fail. That is not the interesting number, and we would not report it on its own. Building the current suite meant first discovering that several of its checks could not fail. The learning-rate check was existential where the specification is universal: it passed when one file in a glob matched, so one stack's check passed while four sibling files carried other learning rates, citing as its evidence a file the campaign does not execute — a reader who set `lrAdam = 1e-2` and `batchSize = 999` in the classes that do run still got `[ok]`. A guard on the output directory could fail only in the case where the check before it had already failed. And the device check verified that a definition string appeared in a source file, which a binary with no CUDA library linked into it satisfies perfectly. The first is universal now and the second resolves both paths against a temporary directory, and each fails on the defect that motivated it; the device clause keeps its source check and has gained an artefact check beside it. We report this rather than quietly regenerating a count, because a specification whose checks are relaxed until they pass is a description of intent again — and a check that cannot fail is the same thing with a green light on it.

The limit is sharper than that, and the campaign is what found it. The suite proved that the architectures matched, that the pixels matched and that the protocol matched, and none of that requires the code to run. Fifteen hours into the campaign 4 runs had failed and all of them were the same ecosystem and architecture: JAX training VGG-16 raised `flax.errors.InvalidRngError` on its first batch, because the shared classifier head carries a dropout layer and that stack's training step never handed it a random stream. The parameter count it had already asserted was correct, since dropout has no parameters, and every check we had added for this campaign passed. A conformance suite built from source text cannot establish that a stack takes one step; only running it can. We fixed the defect and replayed those runs (Section 4.2), and we keep the lesson rather than the embarrassment: static enforcement is a floor, not a ceiling, and the campaign is part of the apparatus that checks the campaign.

## B.4 Residual, documented divergences

Three divergences could not be removed and are reported rather than hidden. First, Deeplearning4j 1.0.0-M2.1 is the last release of its API and links against the CUDA 11.6 runtime, so the Java stack cannot be brought onto the CUDA 12 toolchain the other six share. Second, and more consequentially, that stack resolves its convolutions without cuDNN and cannot do otherwise at this version: they go through nd4j's own im2col and cuBLAS. `readelf -d` on `libnd4jcuda.so` shows a `NEEDED` entry for cuBLAS and none for cuDNN, and the cuDNN convolution helper has no release at any version, so nothing in the stack can call the cuDNN libraries its own redistributable

ships. The specification document in the replication package carries the rest of the evidence. It is the only stack of 7 that reaches no cuDNN kernel, and a conformance check now watches that library and fails if a future version gains one, so the claim cannot quietly stop being true. It cannot be fixed, and we have one reference point for what it may cost: disabling cuDNN in PyTorch, holding hardware, model and data constant, costs 13.07× the energy and 16.42× the time on ResNet-18 at batch 128. That is PyTorch's own non-cuDNN fallback, not nd4j's im2col path, so it neither estimates nor bounds this stack's penalty; it shows only that a missing convolution backend can cost a great deal, so Java's cost in this study may in part reflect that backend rather than a language, and we repeat the qualification wherever that cost is reported. Third, the R `torch` binding cannot switch a script module between training and evaluation mode, so R alone cannot consume the shared TorchScript module of S1 and builds the same architecture instead; its shapes are verified against the other six inside the campaign environment, which is the only place its binding loads. All three are properties of the ecosystems, and arguably part of what one measures when one measures an ecosystem; we name them because a reader deciding what a cost should be attributed to needs to know which of them are in play.

## B.5 Measurement procedure, in detail

The reported energy is the sum of the NVML and RAPL readings (Section 4.2). Its two halves are not at the same boundary, and the sum is not "the chip". NVML's register is a *board* quantity: it includes the GPU die, its GDDR6X and the card's voltage regulators. RAPL's package domain is the CPU package alone. Host DRAM is in neither — this platform exposes no `dram` RAPL domain, so its energy, of order ten watts, is not measurable here and is absent from every figure we report. We name the quantity board-and-package rather than chip-level, and we note the asymmetry with our own criticism of the estimator's modelled RAM term: that term is unverifiable, and ours is missing altogether.

`CodeCarbon` (Courty et al. 2026) runs over the same window as a second reading, in machine tracking mode at a one-second sampling interval and the default `api_call_interval` of 8, configured identically for all 7 stacks through a single shared bridge. We name that last parameter because Section 6.4 shows it is the one that decides whether a reported duration is a duration. The four non-Python ecosystems drive that bridge over a line protocol with synchronous acknowledgement, so that a phase boundary in the measured process and the corresponding counter read cannot drift apart; Section C.4 says what an asynchronous version of it cost.

**Replication and the unit of analysis.** Each of the 42 configurations is executed 5 times as an independent process with a distinct seed, giving 210 runs. 206 of them ran *interleaved* rather than consecutively, so that drift in machine state — thermal, background, driver clock behaviour — is spread across conditions instead of aliasing onto whichever ecosystem happened to run last. The remaining 4, all JAX on VGG-16, are the runs the dropout defect above stopped at their first batch; they were replayed 2 days later on the same host, and they are the one place where interleaving does not hold. Section 9 reports what a between-window calibration says that is worth. All uncertainty we report is between-run: the 30 epochs within a run share

an initialisation, a JIT outcome, an allocator state and a thermal trajectory, and treating them as repeated measurements would understate uncertainty while inflating apparent significance.

**Reproducibility and rigor.** Each run receives a seed from the run contract, and that seed reaches both the data order and the model initialisation in every stack. Two of the three ways a stack can ignore the seed it is handed are now checked against the stacks' own source, which is two more than the audited campaign checked: in that campaign 3 stacks of 7 were handed a distinct seed per repetition and did not use it — one hardcoded its shuffle seed, so all 5 repetitions saw the same data order; one never forwarded the seed to its loader; and one rebuilt its generator inside the epoch loop and replayed a single permutation for every epoch — while the conformance check reported five distinct seeds, having asked the campaign planner rather than the stacks. Initialisation is no longer each framework's default either: under S1 every stack draws convolutions and dense layers from the same distributions, because a default initialiser is not a property of the ecosystem this study is asking about. Nevertheless, full determinism cannot be guaranteed across all stacks due to non-deterministic GPU kernels and differences in backend execution engines; this is one reason the design replicates rather than relying on a single execution per configuration.

A complete replication package is provided (see replication package link in Section 1), including: (i) source code for all implementations, (ii) scripts to reproduce runs, (iii) environment specifications (e.g. virtual environments, Maven, CMake), and (iv) collected raw data (Cruz et al. 2025; Verdecchia et al. 2023). Configurations are standardised across languages where feasible, with deviations documented when framework-specific requirements arise (Pereira et al. 2021; Gordillo et al. 2024). This package enables replication, validation, and extension under different hardware or software conditions (Feldt and Magazinius 2010; Sjøberg and Bergersen 2023).

# C Experimental execution, in full

This section gives the execution detail that Section 5 condenses.

## C.1 Platform

Section 5 describes the workstation.

Two properties of this platform bear directly on the measurements. First, both energy counters are available: the RTX 3090 exposes NVML's accumulated-energy register, and the Ryzen's package-energy counters are readable through the Linux `powercap` interface — which, for historical reasons, presents AMD domains under `intel-rapl` names. We read the package domain and handle counter wraparound at `max_energy_range_uj` explicitly. Second, this is a desktop rather than a server: it has one accelerator, one CPU socket and an order of magnitude less RAM than a datacentre node, which is precisely why the modelled RAM term discussed in Section 6.5 is a smaller share of a software estimator's total here than it would be on a large-memory host.

We report this as a replication platform rather than as the platform. The campaign behind an earlier version of this study — the *audited campaign*, as we call it

throughout — ran on a dual-socket Intel Xeon Gold 5418Y server with an NVIDIA L40S; absolute energy figures are not comparable between the two machines, and none of the comparisons in this paper is made across them. What carries over is the design, the specification and the instrument characterisation, all of which are properties of the method rather than of the host.

## C.2 Model implementations and toolchains

Every ecosystem trains the same architecture, and the stacks whose backend permits it train the same artefact. Under specification S1 the 3 stacks on the pinned LibTorch build — Python/PyTorch, C++ and Rust — do not build 3 independent ResNet-18s and VGG-16s but load one TorchScript module per (architecture, dataset) pair, exported once by a single script and recorded in a manifest with its parameter count and a hash of the parameters in name order. The other four build the same definition in their own framework and assert that count at startup against the same manifest, as does PyTorch; C++, and Rust in the $32 \times 32$ campaign, do not (Section 4.1), and a parity script compares all 7 stacks on the sorted multiset of their parameter tensor shapes rather than on a total. That last distinction is not pedantry. In an earlier campaign of this design the stacks were compared on parameter counts that nothing checked, and VGG-16 ran as four different networks spanning an order of magnitude in parameters under one name; comparing them was comparing models, not ecosystems.

Table 1 lists the versions. The Python stacks are installed in three separate virtual environments rather than one, because their CUDA and cuDNN wheels conflict: a shared environment resolved to a cuDNN that TensorFlow could not dlopen, and TensorFlow then ran unaccelerated while reporting only a warning — a failure mode the accelerator assertion of S4 now turns into an error at startup. The assertion by itself was not enough. A binary can satisfy any source-level device check and still have no CUDA library linked into it, which is what happened to our own Rust stack for a day, so S4 is checked against the built artefact as well: `readelf -d` on every released Rust binary, which immediately found two stale ones.

*Toolchain divergence, and what is left of it.* The 3 stacks of the control group are pinned to one LibTorch build (2.7.0, CUDA 12.8), so within it the backend is genuinely held constant. Four of the 7 ecosystems cannot join them, for three different reasons. TensorFlow and JAX bring their own backends and are not LibTorch stacks at all. Deeplearning4j 1.0.0-M2.1 links against the CUDA 11.6 runtime, resolves its convolutions without cuDNN, and has had no subsequent release; the stack now carries its own redistributables through `org.bytedeco:cuda-platform-redist`, which makes it reproducible without making it contemporary, and Section 4.1 gives one reference point for what the missing convolution backend may be worth. R's `torch` 0.17.0 bundles its own LibTorch and resolves CUDA at load time. We report these as documented divergences rather than normalising them away, and Section 6.2 reports the LibTorch control group separately so that a reader can see how much of the total spread survives when the backend is held fixed.

During a preliminary pilot phase, the Julia programming language (using `Flux` and `Lux`) was also considered. However, persistent technical challenges encountered during implementation related to gradient updates during training, which remained

unresolved also after directly contacting the developers of the libraries, prevented its reliable inclusion in the final study.

## C.3 The saturation cell as executed

Specification S7 adds one contrast cell, executed after the campaign and on the same host: Imagenette[1], ten easily separated ImageNet classes, 9 469 training and 3 925 test images, resized once offline by the shorter side to 224 and centre-cropped to $224 \times 224$ so that, as in the campaign, no stack's resampler enters the comparison. Three rows of Table 2 differ for the cell: input resolution; batch size, 32 in both phases, because VGG-16 at $224 \times 224$ does not fit the card's 24 GB at batch 128 — a hardware limit, not a preference; and evaluation, which still batches at the training batch size but covers the 3 925-image test split. The classifier has ten outputs, as on Fashion-MNIST, so the parameter counts the stacks receive are that pair's. The cell is 7 ecosystems $\times$ 2 architectures $\times$ 5 repetitions, 70 runs of 30 epochs, interleaved repetition-major under the same fixed shuffle. S7 allows the loader-worker count to be raised at this resolution and requires it to be recorded per run; it was not raised, and every run of the cell records the same 2 workers as the campaign. The cell lives in a campaign directory of its own: the driver refuses to schedule it into the campaign's directory or alongside the $32 \times 32$ datasets, the campaign's directory and every table derived from it are frozen, and the analysis writes the cell's tables to a separate location. The two differ in resolution, batch size and class count, so no figure in this paper aggregates across them; Section 6.6 reports the cell as a contrast, and every comparison there is computed the same way on both sides.

## C.4 Measurement and the shared bridge

Each phase of each epoch is bracketed by two counter reads — NVML's accumulated-energy register and the CPU package RAPL counter — and is simultaneously tracked by `CodeCarbon` v3.3.0 (Courty et al. 2026). The two are started and stopped by the same code path, so the intervals they *accumulate over* cannot drift apart. The counter reads are nested immediately inside the tracker's, which costs a fixed offset we quantify in Section 6.5 rather than assume away. The interval CodeCarbon *reports* is a different thing again, and Section 6.4 is about that.

The four non-Python ecosystems drive the instrumentation through a single shared bridge: a Python process that accepts `START`, `STOP`, `METRIC` and `EXIT` over a line protocol, reads the counters, and acknowledges each command before the measured process proceeds. The acknowledgement is not incidental. An earlier, asynchronous version of the same bridge allowed a phase boundary in the measured process to run ahead of the corresponding counter read, which under-reported one stack's training energy by 24% and recorded its evaluation energy as exactly zero — a value that looks like a bug only if someone thinks to look at it.

Running the bridge out of process also isolates the measured stack from the instrument: a crash in the tracker cannot take down a six-hour training run. It does not

[1] https://github.com/fastai/imagenette

remove the tracker's own cost from the measurement — under whole-machine tracking its once-per-second sampling runs inside the block and its package energy is inside the block's RAPL delta. That cost is a constant per second and identical for every stack, so it does not bias the comparison, but it is inside the numbers and we would rather say so. All runs execute in isolation on an otherwise idle host, as whole-machine tracking requires, and each block is logged with dataset, model, ecosystem, repetition, seed, phase, epoch, both energy readings, the duration and the test accuracy reached.

## C.5 Reproducibility artefacts and their limits

A comprehensive replication package accompanies this study (see the link in Section 1). Beyond source code, automation scripts, per-ecosystem environment specifications and raw logs, it contains three artefacts specific to this design:

(i) the **experiment specification** of Section 4.1, written before the campaign and versioned with it;
(ii) the **conformance checker**, 110 executable checks that read the source of every ecosystem and the configuration of every tracker, and fail when a stack drifts out of specification;
(iii) the **dual-instrument records**: every block's counter reading and estimator reading side by side, which is what makes the analysis of Section 6.5 reproducible rather than anecdotal.

Every quantity this manuscript reports from the campaign is generated from those records by the analysis pipeline and injected into the source at build time. The exceptions are stated where they occur: figures recovered from the audited campaign (the boundary contrast of Section 7.1 and the protocol of Table I.5), and one-off diagnostics reproduced from the record that established them (the initialiser trials of Section D.3). Two limits on that provenance are worth stating rather than leaving for a replicator to find. Every run's manifest records the machine state it ran under — driver, power limit, clocks, persistence mode, governor, boost and the precision policy in force — but every run of this campaign predates the harness-provenance field and so carries machine state without a source revision; the field is recorded from the next campaign on. And the analysis used to write its tables into the directory the manuscript reads, so a monitoring pass over a campaign still in flight briefly replaced the campaign's own run count with a partial one; the pipeline now diverts to a separate directory until the expected number of runs is complete. Neither changes a number reported here, and both are the kind of thing a reproducibility claim has to be able to survive being asked about.

The replication package is itself an output of the pipeline and is built at the end of it, so it cannot be older than the tree it flattens — which it had quietly been, for a fortnight, while part of the analysis read it as an input. It refuses to build from a campaign that is not complete, on two conditions rather than one: the campaign must have produced every run it planned, and every run directory present must be whole. A package that silently omits or silently includes runs is worse than one that will not build. A restore script rebuilds the original run tree from the package and diffs it file

**Table D.1**: Mean training energy per epoch (J), accelerator plus CPU package.

| Ecosystem | ResNet-18 | | | VGG-16 | | |
|---|---|---|---|---|---|---|
| | F-MNIST | CIFAR-100 | Tiny-IN | F-MNIST | CIFAR-100 | Tiny-IN |
| Rust | 1 254 | 1 193 | 2 948 | 3 021 | 2 660 | 5 918 |
| C++ | 1 080 | 906 | 1 838 | 2 833 | 2 355 | 4 768 |
| PyTorch | 1 358 | 1 178 | 2 359 | 2 917 | 2 426 | 4 898 |
| JAX | 1 132 | 1 058 | 1 996 | 2 471 | 2 094 | 4 044 |
| TensorFlow | 1 474 | 1 274 | 2 559 | 2 433 | 2 033 | 4 164 |
| R | 10 563 | 8 792 | 17 087 | 9 852 | 8 228 | 16 646 |
| Java | 9 488 | 7 904 | 15 848 | 17 910 | 15 029 | 29 855 |

by file, so the claim that the package is a copy is itself checkable, and every field is carried as text, because parsing a float and writing it back is not a copy.

### C.6 The configuration, and what verifies it

The configuration is in Table 2 of the article. Of its rows, the learning rate and the batch size are verified against the source of every file the campaign executes, universally rather than by one file that happens to match; the precision policy is verified as a campaign-wide export and by a negative check that no stack pins it privately; the other rows are asserted by the specification and were checked by reading. Section B.3 reports why that distinction is not pedantic.

## D RQ1: supplementary analyses

Section 6.2 answers RQ1 and summarises the analyses given in full here: the training energy behind Figure 2, what the superseded campaign's control group seemed to show, the runs that collapsed in the superseded campaign, and the precision policy that separated it from the reported campaign.

### D.1 Training energy per epoch, tabulated

Table D.1 gives the means that Figure 2 plots, per ecosystem and cell; Table 3 in the article picks out the cheapest and the dearest stack of each cell.

### D.2 What the superseded campaign's control group seemed to show

Read from the superseded campaign, the comparison of Table 4 invites a mechanism, and the mechanism is wrong. The argument runs that whatever separates those three cannot be the kernels, because the kernels are the same code, and it puts about half of the ResNet-18 log spread under binding overhead, data movement and host-side dispatch. Sharing a build is not sharing a kernel configuration. Specification S4 then asked for fp32 with TF32 disabled, and exactly one stack of 7 received it: TensorFlow was handed a mixed-precision policy, Java reaches no cuDNN kernel, and the rest took their framework's default, which lets cuDNN use TF32 on this card — so the

superseded and reported campaigns hold that ablation between them. Python/PyTorch's own ResNet-18 accelerator energy per training epoch is 3.14× higher in the superseded campaign, which denied it TF32, than in the reported one, on CIFAR-100 — the one cell the two campaigns leave comparable — while the 3 stacks of that cell whose precision policy did not change move by at most 1.3 %; an isolated kernel probe puts the same flag pair at 4.04× on the accelerator alone (Section D.4). The PyTorch-versus-C++ gap the superseded campaign measured there, 3.94×, is 1.25× in the reported one, and it was nearly absent on VGG-16, whose classifier is GEMM rather than convolution — which no account of binding overhead predicts. Two further premises were false: the three stacks did not begin from identical parameters (Section D.3), and the stacks outside the group were not all running the same network under the name VGG-16. All of that is fixed in the campaign reported here. What we report from Table 4 is the spread that survives holding the model and the backend build fixed. We do not decompose what is left, because a black-box design of this shape cannot, and the cost of guessing is on the record above.

## D.3 Runs that consumed the budget and learned nothing, and why none did here

Replicating each configuration surfaced something a single run cannot show. In the superseded campaign VGG-16 did not always train. We call a run *collapsed* when its final test accuracy never exceeds 1.5× chance for its dataset; the multiplier is generous on purpose, because every collapsed run sat at chance exactly — 1/100 on CIFAR-100, 1/200 on Tiny ImageNet — so no case depends on where the line is drawn. By that criterion 12 of 105 VGG-16 runs collapsed and stayed collapsed for all 30 epochs, having consumed the full energy budget while learning nothing. ResNet-18 never does this in either campaign — 0 of 105 runs in the campaign reported here — and neither architecture does it on Fashion-MNIST.

The per-epoch traces say precisely what those runs were. In every one of them the loss sat at $\ln N$ for the $N$ classes of the dataset — 4.605 on CIFAR-100, 5.298 on Tiny ImageNet — to within 0.0007, for all thirty epochs. That is the loss of a *uniform* output distribution: the network is not confidently predicting one class, it is producing the same distribution for every input, and no gradient signal is reaching it. VGG-16 as shipped carries no batch normalisation, and a plain sixteen-layer network driven by Adam at $10^{-4}$ over a hundred or two hundred classes can start there and never leave.

What decides whether a run does so is the initial weights. It is tempting to argue the opposite: that the 3 stacks described as loading a byte-identical exported module began from identical parameters and still disagreed about which repetitions collapsed, so the deciding factor must be the stochastic path. The premise is false. They did not begin from identical parameters. Python's PyTorch harness built torchvision's network fresh and loaded no module at all, and the C++ and Rust harnesses each loaded a module from a separate, unseeded export.

A controlled side experiment outside the campaign reverses the reading. VGG-16 on the many-class datasets, one framework, 6 runs per initialiser, with framework, optimiser, learning rate and data order held fixed and only the initialiser varied: 0 runs collapse under He initialisation, and 2 and 4 under two variants of Glorot initialisation

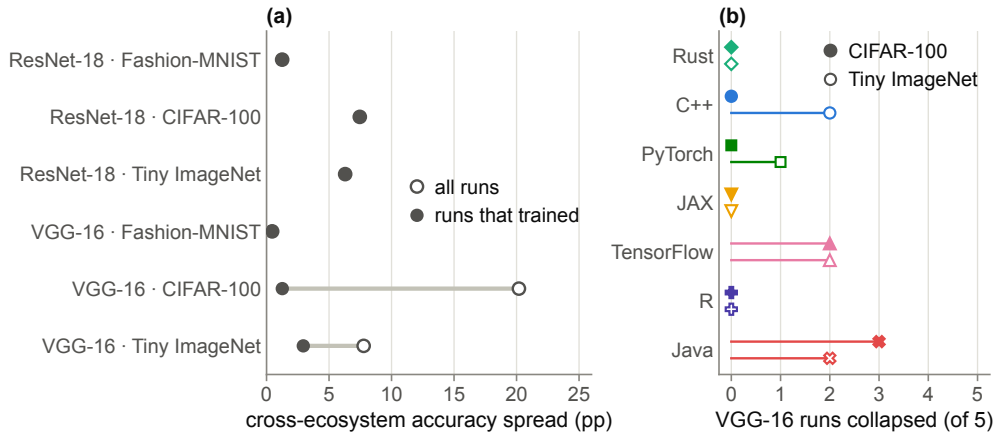



**Fig. D.1**: The superseded campaign, from that campaign's own records. (a) Cross-ecosystem accuracy spread per cell, over all runs (hollow) and over only the runs that trained (filled); the gap on VGG-16 is the collapsed runs. (b) VGG-16 runs that collapsed to chance, out of 5 per cell, on CIFAR-100 (filled) and Tiny ImageNet (hollow). The same figure for the campaign reported here is not drawn: with 0 collapses its panel (a) plots one set of runs against itself and its panel (b) is empty in every cell.

(the same family, also called Xavier; the trial record labels them "Glorot" and "Xavier" without recording which of the uniform and normal forms each was). The stacks were not drawing from one distribution and diverging afterwards; they were drawing from different distributions. Deeplearning4j, the one stack with a hand-rolled initialiser — a truncated normal scaled by fan-in where torchvision scales, untruncated, by fan-out, 4.6× wider at ResNet-18's 7×7 stem — carried 5 of the 12 collapses. Those runs belong to neither campaign and no analysis script regenerates them; they are preserved as a record in the replication package. What we reported as a property of the ecosystems is a defect class about framework defaults: VGG-16 without batch normalisation under Adam at $10^{-4}$ is initialisation-critical, and frameworks ship different initialisers under the same architecture name. An architecture name does not fix the network that gets trained, and nothing in an energy table records which distribution a stack drew from.

The campaign reported here removes the cause rather than the symptom. Every stack starts from one canonical seeded export of each architecture, and the stacks that build their own layers use initialisers matched to torch's. Under that change VGG-16 collapsed 0 times in 105 runs. Zero events do not prove a zero rate: over the 70 runs in the cells where collapse ever occurred — VGG-16 on the two many-class datasets — the exact one-sided 95 % upper bound on the rate is 4.2 %, against the 17 % measured on the same cells before. And with no collapse in any ecosystem there is no table of rates to test: the exact test of homogeneity returns $p = 1.0000$ because every cell is zero, which makes it a statement about the table's shape rather than a test (Section J says why that test replaced the two we first ran).

**A failed recipe and a broken pipeline look identical in an energy table.** Both produce chance accuracy at full energy cost, and neither is visible in a Joule

figure. The per-epoch traces separate them. In a genuine collapse the training loss does not move either, because a network that has settled on a uniform output has nothing left to fit. A pipeline defect looks the opposite: the training loss falls normally while test loss *rises* and accuracy stays at chance, which says that training and evaluation are not seeing the same inputs.

The diagnostic as we first built it could not do that for every stack, and we did not notice. It conjoins the two arms, and our Java harness recorded `NaN` for test loss in every one of its epoch rows; a comparison with `NaN` is false, so the conjunction could never fire for Deeplearning4j — the stack carrying 5 of the 12 collapses. We described that as a diagnostic degrading to its remaining arm. It did not degrade: Java's sensitivity to pipeline defects was zero rather than reduced, and no Java run was ever tested by it. Test loss is now computed in the same inference pass as accuracy, so both arms are live in all 7 stacks and Java's evaluation blocks no longer measure twice the work.

We report this because the discriminator found a defect of our own that the collapse count alone would have absorbed. 4 runs met the chance-accuracy criterion — at most 14.7 % — with training loss falling by 86–87 %: one Rust binary carried a private copy of the input transform, applied on the training path only, which resized the tensor its loader had already produced using a function that returns `uint8`. Every value in $[0, 1]$ truncated to zero, so it trained on black images and was evaluated on real ones. Filed as collapses they would have read as "VGG-16 sometimes fails to train". Those runs were deleted and the configuration re-executed; since the evidence for a defect should be as traceable as the evidence for a result, and the discarded runs no longer exist in the campaign, their signature is preserved as a record in the replication package rather than quoted from memory.

The same defect class came back later in a different file, and what caught it was not this diagnostic. When the datasets were pre-resized offline so that each stack's own resize became a no-op, the `uint8` guard was written into one of the three Rust loaders and not the other two, so the Fashion-MNIST loader went on resampling an already 32×32 image and its pixel statistics stayed away from the other stacks' — not far enough to reach chance accuracy, and so invisible to a test that triggers on chance accuracy. It was found by comparing pixel statistics across all 7 stacks. All three loaders are guarded now. In the campaign as it stands the discriminator finds 0 pipeline defects and 0 collapses, which is what a campaign with nothing to separate looks like rather than a claim that the class is closed.

We are able to state the discriminator's recall on the one instance we have, and it is not perfect. The defect was a deterministic code path on the training branch, so all 5 repetitions of that configuration carried it; the discriminator flagged 4. The remaining run finished above the collapse threshold and so was never a candidate for the test at all, and the second instance of the same defect class — the loader described above — never came near the threshold either. That is precisely why a discriminator triggering on chance accuracy is a diagnostic aid and not a detector. One calibrated on a single positive example, with a threshold chosen after seeing it, is a heuristic: it separates the two failure modes cleanly once a run is known to have failed, and it will miss a defect that leaves enough accuracy behind. We report it as the diagnostic aid it is.

Two consequences matter here.

**The stacks agree on what they compute.** No cell shows a cross-ecosystem accuracy spread above 6.5 percentage points, and the widest of them, on ResNet-18 on CIFAR-100, is a spread between runs that all trained. In the superseded campaign the same measurement had two arms — over all runs, and over the runs that trained — and almost the entire apparent disagreement between stacks sat in the gap between them (Figure D.1a). There is no gap now: every run in this campaign trained, the two arms coincide, and the spread is what the stacks compute rather than what failed to train. Pooled over both architectures — the one place we quote a pooled accuracy, because the comparison here is with and without collapsed runs rather than between stacks — the CIFAR-100 spread is 4.0 points either way.

**A collapse breaks fixed-budget energy comparison, invisibly.** A collapsed run costs full energy and produces nothing: in the superseded campaign 10.7 % of the training energy went on runs that learned nothing, against 0.0 % of the 36.0 MJ spent here. An ecosystem whose default initialiser differed from the recipe's looks equally expensive and much less accurate; one whose default matched it looks efficient. With a single run per configuration — the design this study replaces — the reported number is whichever outcome that one configuration produced, and nothing in the energy data distinguishes the two cases. This is why we report accuracy alongside energy rather than assuming a fixed budget delivers fixed quality.

## D.4 The precision policy, and what it was worth

The present design was executed twice, and the difference between the superseded and the reported campaign is a measurement. The superseded campaign pinned TF32 off — `cudnn.allow_tf32` and `matmul.allow_tf32` false, `cudnn.deterministic` true — in the harness the Python stacks import. Only those three stacks import it, and of those only PyTorch reached cuDNN through it: TensorFlow was handed a mixed-precision policy, which is a different axis, and JAX was handed nothing. Java reaches no cuDNN kernel at all, and the remaining stacks took their framework's default, which lets cuDNN use TF32 on this card. So one stack of 7 trained true-fp32 convolutions. The reported campaign runs all 7 under the single policy of Section 4.1. Over the same 42 configurations, the two campaigns are therefore an ablation of PyTorch's policy against a control group — C++, Rust and R — whose precision policy did not change, and we report them as one; TensorFlow's policy changed too, and it is excluded on other grounds below. (The audited campaign that motivated this design, in Section 8, is a different thing again and ran on different hardware; nothing here is compared against it.)

Table D.2 puts the two campaigns side by side, in accelerator energy per epoch of training, and says of every ResNet-18 training cell whether anything other than the precision policy changed between them. On CIFAR-100, where nothing else did, Python/PyTorch's ResNet-18 accelerator training energy is 3.14× higher with TF32 denied than with it allowed, while the 3 stacks that are comparable beside it — C++, Rust and R, whose precision policy did not change and whose remaining differences between the two campaigns cannot move per-epoch energy — move by at most 1.3 %. And the quantity that most invites a binding-overhead reading, the Python/PyTorch-versus-C++ training gap on ResNet-18, falls from 3.94× to 1.25×. Each side of that

**Table D.2**: Mean accelerator (GPU) energy per training epoch, ResNet-18, in the superseded campaign (TF32 denied for Python/PyTorch alone) against the reported one (TF32 allowed for all seven). Five repetitions and 150 epochs per cell on each side. **Python/PyTorch on CIFAR-100 is the contrast the text cites**; the cells above the rule are the ones the two campaigns leave comparable, and each cell below it is excluded for the stated reason rather than for its value.

| Ecosystem | Dataset | TF32 off (J) | TF32 on (J) | Ratio | Comparable |
|---|---|---|---|---|---|
| Rust | CIFAR-100 | 975 | 968 | 1.01× | yes |
| C++ | CIFAR-100 | 741 | 742 | 1.00× | yes |
| **PyTorch** | CIFAR-100 | 2 919 | 930 | **3.14**× | yes[a] |
| R | CIFAR-100 | 6 064 | 6 147 | 0.99× | yes |
| Rust | Fashion-MNIST | 1 084 | 1 027 | 1.06× | data[b] |
| C++ | Fashion-MNIST | 890 | 890 | 1.00× | data[b] |
| PyTorch | Fashion-MNIST | 3 509 | 1 077 | 3.26× | data[b] |
| JAX | Fashion-MNIST | 538 | 801 | 0.67× | window[c] |
| TensorFlow | Fashion-MNIST | 927 | 1 139 | 0.81× | network[d] |
| R | Fashion-MNIST | 8 154 | 7 390 | 1.10× | data[b] |
| Java | Fashion-MNIST | 7 937 | 7 518 | 1.06× | network[e] |
| JAX | CIFAR-100 | 480 | 742 | 0.65× | window[c] |
| TensorFlow | CIFAR-100 | 777 | 980 | 0.79× | network[d] |
| Java | CIFAR-100 | 6 611 | 6 265 | 1.05× | network[e] |
| Rust | Tiny ImageNet | 3 379 | 2 367 | 1.43× | data[f] |
| C++ | Tiny ImageNet | 1 703 | 1 511 | 1.13× | data[f] |
| PyTorch | Tiny ImageNet | 5 874 | 1 871 | 3.14× | data[f] |
| JAX | Tiny ImageNet | 1 354 | 1 414 | 0.96× | window[c] |
| TensorFlow | Tiny ImageNet | 1 884 | 1 980 | 0.95× | network[d] |
| R | Tiny ImageNet | 12 630 | 11 961 | 1.06× | data[f] |
| Java | Tiny ImageNet | 13 354 | 12 566 | 1.06× | network[e] |

[a] PyTorch's harness also moved from eager torchvision to the exported TorchScript module between the executions, so this ratio bounds the precision effect from above.
[b] Fashion-MNIST training images were re-encoded 28 → 32 on disk between the executions, so every stack's loader does different work.
[c] Python/JAX was measured through a window that did not force completion in the first execution and does in this one.
[d] Python/TensorFlow ran a Model Garden ResNet-18 with a projection shortcut the other stacks do not have, and dropped its last partial batch.
[e] Java/DL4J built its graph by hand, with truncated convolution padding and a bias on every convolution.
[f] Tiny ImageNet training images were re-encoded 64 → 32 on disk between the executions, so every stack's loader does different work.

comparison is 5 runs of thirty epochs, on real data, on the instrument this paper uses throughout.

Most cells are not a precision contrast, and the table says which and why. Three of the other stacks are excluded for reasons of their own: Python/TensorFlow ran a Model Garden ResNet-18 with a projection shortcut the other stacks do not have and dropped its last partial batch, Java/DL4J built its graph by hand with truncated convolution

padding and a bias on every convolution, and Python/JAX was measured through a window that did not force completion in the superseded campaign and does in the reported one. VGG-16 is excluded entirely, since the superseded campaign ran four different networks under that name, so a VGG-16 ratio between the two campaigns is mostly architecture. Fashion-MNIST and Tiny ImageNet were re-encoded to the common $32 \times 32$ resolution on disk between the two, so on those datasets every stack's loader does different work; CIFAR-100 was already at that resolution and did not move. One caveat survives inside the cells that remain, and we state it rather than remove it: the superseded campaign built PyTorch's ResNet-18 fresh from torchvision and the reported one loads the exported module of S1, so PyTorch's ratio contains whatever TorchScript execution is separately worth. The control group cannot separate the two, because the other stacks loaded a module in both campaigns. The ratio therefore bounds the precision effect from above, and we do not partition it. The kernel probe below reports a larger ratio for the same flag pair, and the two are not in tension: the probe measures accelerator energy per step on a resident batch, where this figure is accelerator energy per epoch of real training, loader and data movement included.

Underneath that contrast we ran a kernel-level probe on an idle host (`scripts/probe_tf32.py`): ResNet-18 built from the definition the campaign exports, batch 128, one resident batch of synthetic input, and the flags set directly. Synthetic input is the point rather than a compromise, since which convolution algorithm cuDNN selects does not depend on what the pixels are, and holding one batch resident takes the loader out of the comparison identically in every cell. Every ratio here is against the configuration the stacks on their framework default ran in the superseded campaign: TF32 allowed to cuDNN, determinism unpinned, the matrix-multiply path at its framework default of fp32 — the cuDNN-disabled cell excepted, which is taken against this campaign's own default, the only configuration whose matrix-multiply flag it shares. A thousand training steps per measured window (three hundred for the cuDNN-disabled cell) clears the 5 s floor the script enforces against an NVML energy register that advances about ten times a second on this device (Section 6.5), so these ratios are not quantisation-bound; the cells are reshuffled from a recorded seed within each repeat, so drift over the run cannot alias onto one of them, and we report medians over the repeats. Denying TF32 to the convolutions costs 3.62× the accelerator energy per step. Denying it and pinning `cudnn.deterministic` as well — the superseded campaign's configuration for PyTorch, exactly — costs 4.04×. Determinism on its own costs 0.99×, which is to say nothing we can measure; it is pinned nowhere in this campaign, and it is not expressible in the Deeplearning4j or R bindings at all. Allowing TF32 in the matrix-multiply path on top of the convolution path changes nothing we can measure on ResNet-18 either, because the network's only general matrix multiply is the $512 \to 100$ classifier head. One further cell disables cuDNN altogether, as one reference point for what Deeplearning4j's absent cuDNN path may cost; that belongs with the cost of the Java stack rather than here. And the probe diverges from the campaign in one respect, which we state because it caps what the probe can be used for: it runs the eager module rather than the TorchScript export the campaign executes, so it measures cuDNN algorithm selection under the eager path.

That the matrix-multiply path is where ResNet-18's arithmetic does *not* go is also why the superseded campaign showed nothing on VGG-16. Python/PyTorch, C++ and Rust all ran torchvision's VGG-16 head in that campaign, so a comparison between them within it is like-for-like even though a comparison of VGG-16 across the two campaigns is not; and there the PyTorch-versus-C++ gap sat close to one, against 3.94× on ResNet-18. VGG-16's head is GEMM-dominated and the matrix-multiply path ran fp32 in every binding, so the gap disappeared exactly where the workload stopped being convolution.

The gap a study of this shape would naturally call binding overhead — the superseded campaign's ResNet-18 gap between Python/PyTorch and the two stacks sharing its backend — was one configuration flag, in exactly one stack of 7, and we would not have found it without a control group of stacks that could not import the harness the flag lived in. The correction changes the finding's category rather than removing it: what was an undeclared confound is a declared ablation, and it says that a precision policy was worth more here than any difference between bindings this study measures. A cross-ecosystem energy comparison that does not state its precision policy is therefore not comparing ecosystems, whatever else it holds fixed. What remains of the Python/PyTorch-versus-C++ gap once the policy is common to all 7 is 1.25×, and we offer no account of it here.

# E RQ3: the reported-window mechanism

Section 6.4 establishes that the duration CodeCarbon reports is padded in 3 discrete modes and that power derived from it is understated. This section gives the modes of the excess in full, the mechanism behind the padding, the probe that measures it, where in the campaign it lands, and how we arrived at the explanation.

## E.1 The modes of the excess

Panel (a) of Figure 4 plots, for each block, how far the duration CodeCarbon reports exceeds the counter-bracketed phase, against that phase. The excess between them is not continuous: it is concentrated at a few values with wide gaps between them. 25 % of blocks carry an excess of 0.01 s — that is, none — and 75 % carry seconds, with a median of 4.57 s. That second group is two populations rather than one, a second or so apart and 2.08 s across end to end; the blocks scattered between them are why the mode-finder we use, which cuts the sorted excess at gaps wider than half a second, reports the pair as one. 1 of the 12 600 blocks sits far above both, at 13.38 s. These clusters of the campaign's excess are the *modes* of Figure 4a; the cost of `stop()` measured directly, below, comes in three *levels*. The lowest mode is the no-lookup level, the middle mode holds the two padded levels, and the top mode matches no level. 75 % of the campaign's blocks are therefore reported with a window seconds longer than the interval over which their energy was accumulated.

**Table E.1**: Cost of `EmissionsTracker.stop()` against the length of the block it closes, measured directly on this host with no workload (`scripts/probe_reported_window.py`). The three levels are two, one and zero outstanding network lookups. The probe's threshold of 12 s and the campaign's of 11 s are one boundary estimated two ways.

| Block held open | Cost of `stop()` | Lookups outstanding |
|---|---|---|
| 2 s | 4.581 s | 2 |
| 5 s | 4.560 s | 2 |
| 8 s | 4.559 s | 2 |
| 9 s | 3.014 s | 1 |
| 10 s | 3.014 s | 1 |
| 11 s | 3.016 s | 1 |
| 12 s | 0.012 s | 0 |
| 15 s | 0.012 s | 0 |
| 20 s | 0.013 s | 0 |

## E.2 The cause, measured directly

The cause is a network call, and it is reproducible in two minutes. `EmissionsTracker.stop()` resolves cloud metadata and then the host's geolocation over the network — two blocking HTTP requests — *after* the final energy reading. That is why only the duration is affected and the energy is not. Both results are cached on the tracker instance, and a background thread performs the same call once 8 measurements have accumulated. So a block pays for however many of the two lookups are still outstanding when it ends.

Table E.1 is a direct probe on this host, with no accelerator and no workload: a tracker started, held open for a fixed interval, and stopped. Blocks that end before the background call pay 4.56 s; blocks that outlive one of the two lookups pay 3.01 s; blocks longer than 12 s pay 0.012 s. The campaign's excesses are not a mystery about the tracker's lifetime: they are how many network round trips remain, and all three of the probe's levels appear in the campaign. The padded blocks divide between the upper two, close to 3.01 s and 4.56 s, with every stack but R in both; which of the two a block falls in tracks the clock rather than the stack, as we report below. 1 block matches no level of the probe at all, and we flag it rather than explain it.

## E.3 Where the padding lands

This explains the campaign. All 8 942 blocks shorter than 9 s are padded, 100 % of them; the shortest unpadded block in the campaign is 10.8 s and the longest padded one 12.3 s, so the two populations meet only in that interval. A single threshold at 11 s classifies 98.8 % of the 12 600 blocks correctly. It misses 154 — nearly all of them C++ training blocks, which are padded at durations where every other stack is not, and which we cannot account for — and it wrongly predicts padding for 3 blocks that sit just under the threshold. The probe of Table E.1 puts the same boundary at 12 s: one

**Table E.2**: Where the reported window is padded, by stack and phase, with the range of block lengths each stack produces. The pattern follows the threshold of Table E.1; C++ training does not.

| | Training | | Inference | |
|---|---|---|---|---|
| **Ecosystem** | **block length** | **padded** | **block length** | **padded** |
| Rust | 4.4–18.2 s | 67% | 0.4–1.0 s | 100% |
| C++ | 2.9–12.3 s | 100% | 0.3–0.4 s | 100% |
| PyTorch | 4.2–12.7 s | 83% | 0.9–1.0 s | 100% |
| JAX | 4.0–22.5 s | 97% | 0.7–6.7 s | 100% |
| TensorFlow | 3.9–15.6 s | 98% | 0.7–1.2 s | 100% |
| R | 41.6–95.7 s | 0% | 11.9–13.8 s | 0% |
| Java | 31.6–79.8 s | 0% | 4.0–9.5 s | 100% |

threshold estimated two ways, from the campaign and from the probe, which agree that it lies between 11 and 12 s.

Table E.2 shows which stacks this reaches, and the answer follows from block length as the mechanism predicts. Every inference block in the campaign is under the threshold except R's, and every one of those short blocks is padded. Training divides the same way: Java and R, whose training epochs run for tens of seconds, are never padded; the stacks with short epochs almost always are. C++ is the exception in both directions, and we flag it rather than explain it.

Two consequences follow, and the second is the useful one. The magnitude of the padding is network latency on this host, not a constant of the tool: in our records the padded blocks divide by time of day, a median excess of 4.58 s in the night hours against 3.29 s in the day. And the whole artefact is a configuration away from disappearing — an offline configuration, or an `api_call_interval` of $-1$, removes the lookups. A study that measures short phases with this tool and does not set that parameter will record inflated durations, and nothing in the output says so.

## E.4 Power from each instrument's own fields

Table E.3 gives, by phase length, the medians that Figure 4b plots: power derived from the estimator's own energy and duration fields against power from the counters. Both sides are the terms the two instruments both measure.

## E.5 Fitted models of the window, and why the mechanism settles it

A length-based model of the reported window is tempting, and its fit statistics mislead in both directions. A floor, max(phase, 4.94 s), fits at $R^2 = 0.9819$ and is wrong in the middle of the range; two regimes — phase plus a constant below a threshold — fit at $R^2 = 0.9976$ with a third of the error. The three modes and the exceptions above the threshold then suggest that no length-based model can be right.

That conclusion over-corrects. A curve fitted to a predictor can be right for the wrong reason, and it can also be right: what settles it is the mechanism, not the

**Table E.3**: Power derived from the estimator's own energy and duration fields against power from the counters, by phase length. No model is fitted: each instrument is divided by itself, on the terms both of them measure. The last column is the median of the per-block ratios, not the quotient of the two medians beside it.

| Phase length | Blocks | Power from the reported fields | Measured power | Understated by |
|---|---|---|---|---|
| <0.5 s | 1 050 | 20 W | 216 W | 13.03× |
| 0.5–1 s | 3 376 | 40 W | 243 W | 5.69× |
| 1–2 s | 44 | 43 W | 190 W | 5.06× |
| 2–5 s | 2 222 | 143 W | 278 W | 1.97× |
| 5–10 s | 2 408 | 230 W | 366 W | 1.56× |
| 10–11 s | 150 | 314 W | 407 W | 1.30× |
| 11–30 s | 1 550 | 177 W | 177 W | 1.00× |
| >30 s | 1 800 | 217 W | 217 W | 1.00× |

fit statistic, and until we measured `stop()` directly we had no mechanism and were arguing from residuals in both directions. The threshold is real, it has a cause, and the cause predicts where the threshold sits. The general lesson is narrow: a high $R^2$ on a skewed predictor is not evidence of the right functional form, and neither is a scatter of exceptions evidence against one.

# F RQ4: further detail on the two instruments

## F.1 The resolution of the reference, and a fixed offset

Two caveats belong with the energy agreement reported in Section 6.5.

The first is that our reference is not arbitrarily fine either: NVML's accumulated-energy register advances about ten times a second on this device, so a block is quantised to roughly 100 ms of accumulation. At training power that is tens of joules per block, which is why the accelerator terms agree to a median of 1.2 mJ but not to that in the tail. It does not affect the run-level means this paper reports — the quantisation is unbiased and averages over thirty blocks — but a per-block agreement claim on a sub-second phase is at the resolution of the instrument, and the inference figures of Section 6.3 should be read with that in mind.

The second is that one figure we reported must be read carefully. Expressed as a percentage, a fixed offset — a median of 0.49 J on the accelerator-plus-CPU total, against 0.48 J on the CPU term alone — grows without bound as blocks shorten: the mean per-block error is 0.01 % above 30 s and 0.9 % below one second. That is arithmetic on a constant, not a degradation of the instrument, and it does not mark the sampling interval. There is no transition: the absolute residual is flat across every duration decile of the campaign's blocks, from the shortest inference phase to the longest training one.

## F.2 The modelled RAM term

On what is not measured, the two readings cannot agree. CodeCarbon's total exceeds the counters by 1.087×, and the excess is entirely its RAM term — 7.6 % of its reported

total on average — which *in machine tracking mode* — the mode this campaign used, and the one a whole-machine study needs — is derived from installed memory capacity rather than from memory activity. The model is specific: a constant watts-per-DIMM, over a DIMM count *inferred* from total capacity, with decreasing marginal power above four DIMMs and a cap. In process mode it uses the process's resident set instead, so the criticism here is about machine mode and should not be read past it. On this machine it infers four modules from 30.2 GB and yields a flat 20 W for every block of the campaign, regardless of what the workload does to memory. No counter on this platform can confirm or refute it, because there is no DRAM RAPL domain to read. We could not read the machine's memory table to check the inferred count either — that needs privileges the measurement account does not have — and we note only that a desktop reporting this capacity is at least as likely to hold two modules as four, in which case the same model would charge half as much. The objection is not the factor but that the input is a guess about the hardware rather than a reading of it.

The number is not wrong so much as it is not a measurement, and it is added silently to a figure otherwise composed of measurements. On a machine with more installed RAM the model contributes more — bounded, because of the cap, but enough that the reported energy of an identical workload depends on the host's memory configuration. Our own boundary has the opposite failing and we state it plainly: the counters omit DRAM energy altogether. One figure contains an unverifiable term; the other is missing a real one. Neither is a whole-system number, and the difference between them is not an error bar.

## F.3 The time between the phases, and coverage

The time between the phases belongs to the instrument. Energy is attributed only to the phases each stack brackets, and the stacks differ enormously in how much of a run that is. Between the start of a run's first block and the end of its last, tracked time ranges from 46 % (C++) to 100 % (R): more than half of a C++ run's measured interval falls outside any measured block. We state the denominator precisely because it is not the process's wall time — start-up, dataset construction, module loading, warm-up before the first epoch and teardown lie outside it, and are excluded from both sides of the ratio.

The natural reading is that the low-coverage stacks do a great deal of work between phases and are flattered by a per-phase comparison. We pursued that reading and it does not survive. The gap preceding each block is close to the amount by which CodeCarbon's reported window exceeds the counter-bracketed phase: the two correlate at $r = 0.98$ over every block in the campaign, they differ by a median of 0.26 s block by block, and the window excess accounts for 97 % of all untracked time. That pooled correlation is largely a contrast between padded and unpadded blocks rather than a fine agreement, and we report it as such: within the padded blocks alone it is $r = 0.85$, and within the unpadded blocks $r = 0.03$. The timestamps these gaps are computed from have 1 s resolution against a median gap of 3.51 s, and 9 % of the computed gaps come out negative before they are clipped at zero. That fraction, rather than the correlation, is the honest measure of what this attribution can resolve.

**Table F.1**: Tracked share of the interval from a run's first block to its last, and how much of the remainder is the instrument's own window. Not wall time: the denominator excludes start-up, dataset construction and teardown. Coverage follows block length, not the ecosystem — with R the exception, whose untracked time is almost none of it.

| Ecosystem | Median block | Tracked share | Of the untracked rest, the instrument |
|---|---|---|---|
| Rust | 4.3 s | 59% | 100.7% |
| C++ | 3.1 s | 46% | 101.1% |
| PyTorch | 3.9 s | 54% | 100.8% |
| JAX | 5.0 s | 49% | 96.3% |
| TensorFlow | 3.2 s | 48% | 99.8% |
| R | 33.5 s | 100% | 14.3% |
| Java | 25.4 s | 93% | 98.8% |

Coverage tracks block length rather than anything about the ecosystems (Table F.1): short blocks pay the tracker's fixed cost more often, so the stacks with the shortest median block are penalised where those with the longest are not. The untracked time is the instrument holding its window open. It would not exist in an uninstrumented run, and charging it to the ecosystems would charge them for the cost of being measured. Per-phase energy is therefore the right quantity, and the spreads reported in Section 6.2 stand as they are.

Two caveats belong with that conclusion. The attribution is not exact — for 3 of the 7 ecosystems the instrument's share of untracked time comes out slightly above 100 %, which is the one-second timestamp resolution showing through, and Table F.1 prints it unclamped rather than rounding it away. And the untracked time is not free. Across the campaign it amounts to 10.5 hours. Pricing it is a range rather than a number: at this machine's measured idle draw of 64 W it is 2.4 MJ, about 6 % of the energy the campaign reports, and that is a floor — the interval begins microseconds after an epoch ends, with the accelerator not yet clocked down, so the true figure lies between that and the several times larger number the campaign's mean block power would give. Its share grows as blocks get shorter. It is also avoidable, now that we know what it is: it is not an intrinsic cost of machine-mode tracking but of leaving one configuration parameter at its default.

# G The saturation cell: further detail

Section 6.6 reports the contrast cell of specification S7 in condensed form. This section gives its four central paragraphs in full, including the record coverage, the per-pair utilisation gains, the accuracies reached, the control group's spreads and the inference spreads at both resolutions. Table 9 and Figure 6 are in the main text.

## G.1 Accelerator loading

**The cell loads the accelerator.** Utilisation here is the 1 Hz `nvidia-smi` record over each run's training blocks only, on both sides of the contrast; it is a different window from the whole-run figures of Section 9 and is not comparable with them. On

that window the median over the cell's 14 (ecosystem, architecture) pairs is 89.2 %, from 18.1 % (R, ResNet-18) to 99.3 % (JAX, VGG-16), against 5.5 % (R) to 93.8 % (C++) at $32 \times 32$. The 1 Hz record covers 9–15 of the 15 runs of each pair at $32 \times 32$, because it began after the campaign did, and 70 of 70 in the cell. Per pair the change is a median of 12.0 points and ranges from -1.9 to 60.5. Two stacks fall well short of saturation. Rust on ResNet-18 does not rise at all, and R remains the least loaded stack on both architectures — the low duty cycle Section 9 describes, which the cell does not remove. The cell ran at the specification's 2 loader workers, and we did not vary that count, so it does not tell us whether more workers would close R's gap. Every stack learned: the median run of each cell ends at 68.1–76.1 % test accuracy on ResNet-18 and 78.9–80.9 % on VGG-16 over ten classes, and the lowest single run of the cell (Java, ResNet-18) ends at 64.3 %, far from chance.

## G.2 The LibTorch lineage and the control group

**Within the LibTorch lineage the spread compresses.** The four stacks that descend from LibTorch — PyTorch, C++, Rust and R — spread by 9.3×–9.8× in ResNet-18 training over the three $32 \times 32$ datasets and by 2.8× at $224 \times 224$; on VGG-16, by 3.5× and 1.3× (Figure 6). Inference compresses further, from 30.6×–38.7× to 4.9× on ResNet-18 and from 18.9×–19.9× to 2.6× on VGG-16. The accelerator-only spread at $224 \times 224$ is of the same size, 2.4× and 1.3× in training. The compression has one author. C++ is the cheapest member of the four and R the dearest for every architecture and phase, at both resolutions, so the family's spread is R's distance from C++, and that is the distance that closes. That is consistent with a host-side share in R's cost, R being the stack that leaves the accelerator least loaded, but the cell does not decompose R's cost and we do not claim the share's size. The 3 stacks of the control group do not compress in training, where their spread was small to begin with: 1.3×–1.6× on ResNet-18 and 1.1×–1.2× on VGG-16 at $32 \times 32$ (Table 4), against 1.4× and 1.1× at $224 \times 224$. In inference they narrow, from 2.5×–3.0× and 2.1×–2.4× to 1.2× and 1.3×; those $32 \times 32$ inference figures are computed here on the same definition and have no published counterpart, since Table 4 reports training only.

## G.3 All seven stacks, and Java

**Across all 7 the spread widens.** Training energy spreads by 15.3× on ResNet-18 and 19.1× on VGG-16, against 9.3×–9.8× and 7.4× at $32 \times 32$ (Figure 6); the accelerator-only spread is 15.1× and 19.0×, so the widening is not a host-side term. The dearest stack for every architecture and phase in the cell is Java; at $32 \times 32$ the dearest in training was R on ResNet-18 and Java on VGG-16 (Table 3). Setting Java aside, the other six spread by 3.6× and 2.0× in training at $224 \times 224$, against 9.3×–9.8× and 4.0×–4.1× at $32 \times 32$: without Java the spread compresses, as it does within the LibTorch lineage. Inference is mixed. On ResNet-18, where R was the dearest stack at $32 \times 32$, the seven-stack spread narrows, from 30.6×–38.7× to 24.5×; on VGG-16, where Java already was, it widens from 23.1×–23.7× to 48.8×.

Java is the stack of the seven that reaches no cuDNN kernel (Section 4.1), and the qualification attached to its cost there applies here with more force. At $224 \times 224$ its

accelerator is busy on both architectures (Table 9), so its cost in this cell is device work rather than host waiting. That is consistent with the account of Section 4.1, in which a convolution path without cuDNN is what separates Java from the other six. It does not measure it: the reference point of Section 4.1, 13.07×, was taken with PyTorch at $32 \times 32$ on ResNet-18 at batch 128 and we did not repeat it at $224 \times 224$, so we cannot say how much of Java's distance in this cell the missing backend accounts for. Java's cost here is in part a statement about a convolution path, not about a language.

### G.4 The ordering

**The ordering partly survives.** We correlated the ecosystem ordering at $224 \times 224$ with the ordering on each $32 \times 32$ dataset, 12 comparisons in all, with an exact permutation $p$ because seven items give only 7! orderings. VGG-16 training keeps its order, or nearly: $\rho = 1.00$, 1.00 and 0.96 against CIFAR-100, Fashion-MNIST and Tiny ImageNet. ResNet-18 training ($\rho = 0.64$ to 0.79) and inference on both architectures ($\rho$ down to 0.54) agree more weakly, and with seven items the weakest of them ($p$ up to 0.2357) are compatible with no agreement at all. What moves is the top. C++, the cheapest ResNet-18 stack in training on every $32 \times 32$ dataset, is not the cheapest at $224 \times 224$, where the cheapest training stack is JAX on ResNet-18 and TensorFlow on VGG-16. In a single comparison up to 4 stacks move by two places or more. A ranking read off the campaign is a statement about its regime.

## H An industrial simulation of Deep Green AI large-scale impact

This section is a projection, not a measurement, and is kept out of the main text for that reason.

To contextualise our empirical results in a hyperscale setting, we present an illustrative industrial scenario that translates our *relative* ecosystem-level energy differences into approximate energy and cost impacts. This scenario is not intended as a direct estimate for any specific production system, model family, or hardware stack; rather, it shows how small per-workload gaps can compound at scale.

**Relative multipliers.** For a given phase (training or inference) and ecosystem $k$, we define a phase-specific energy multiplier relative to any reference ecosystem *base* as:

$$m_k = \frac{\bar{E}_k}{\bar{E}_{\text{base}}},$$

where $\bar{E}_k$ is the mean energy measured under our experimental protocol for that phase. Accordingly, $m_k$ should be interpreted as a *relative energy-per-work* factor under our setup, not as a universal constant.

Under the simplifying assumption that the ecosystem-level multiplier transfers to the same workload definition, for any baseline energy budget $\bar{E}_{\text{base}}$, the scaled energy is calculated as $\bar{E}_k = m_k \cdot \bar{E}_{\text{base}}$.

**Training phase (illustrative scaling).** From Table D.1, moving from the most expensive ecosystem in a cell to the cheapest changes training energy by 7.4×–9.8×. Taking a concrete case: if a *training* budget is $E = 1$ GWh on the most expensive

**Table H.1**: Projected daily energy and cost for large-scale inference at a fixed service target ($10^5$ baseline accelerators, 24 h, electricity at 100 €/MWh). Fleet size scales with measured accelerator-hours; energy is that fleet at each stack's own measured draw. These are scenario arithmetic on measured quantities, not forecasts.

| Ecosystem | Accelerator-hours | Fleet | Measured draw | Energy (MWh/day) | Cost (€/day) |
|---|---|---|---|---|---|
| C++ | 0.37× | 36 796 | 197 W | 174 | 17 365 |
| Rust | 0.68× | 68 031 | 190 W | 311 | 31 060 |
| TensorFlow | 0.81× | 81 069 | 173 W | 336 | 33 644 |
| JAX | 0.94× | 94 170 | 166 W | 376 | 37 560 |
| PyTorch (baseline) | 1.00× | 100 000 | 167 W | 401 | 40 088 |
| R | 13.22× | 1 321 558 | 121 W | 3 850 | 384 977 |
| Java | 6.06× | 606 078 | 265 W | 3 859 | 385 886 |

stack of a cell, the same fixed-budget run costs between 1/9.8 and 1/7.4 of that on the cheapest. At an electricity price of 100 €/MWh — a round figure in the range of recent European non-household prices, chosen for legibility rather than precision — that budget costs 100 000 € in total, and the spread between the two ends of a single cell is 86 416–89 778 € per GWh of training: not a margin on the bill but very nearly the whole of it. We stress that this is arithmetic on a relative multiplier, not a forecast: it assumes the multiplier transfers to a workload of a different size on different hardware, which is exactly the assumption our own measurements give us the least confidence in.

**Inference phase (hyperscale workload, fixed work).** Let PyTorch serve a fixed daily inference volume on $N = 100{,}000$ accelerators running continuously for 24 hours. We take the per-card draw from the campaign rather than assuming one: PyTorch's inference blocks measure 167 W at the accelerator, giving 401 MWh/day, or 40 088 € per day at 100 €/MWh.

Scaling that to another ecosystem takes two measured quantities, not one. At a fixed service target a less efficient stack needs more *accelerator-hours*, so the fleet scales with the time multiplier; and it draws its own power while doing so, which is not the baseline's. Both matter here, and they do not move together. Java needs 6.1× the accelerator-hours and draws 265 W per card against the baseline's 167 W, so the two compound and its energy multiplier is 9.63×, larger than either factor alone. They do not always compound: in Table H.1 the ordering by fleet size is not the ordering by daily energy, because each fleet is multiplied by its own measured draw. Multiplying energy at a fixed fleet and a fixed per-card draw would assert a draw for Java that its own measurements contradict, and one several times the board limit of the card.

Table H.1 therefore gives fleet size, measured draw and the energy that follows. The multipliers are computed from accelerator energy alone, matching the per-accelerator quantity the scenario scales; at the board-and-package boundary the ecosystem spread is 24.6× rather than 22.2×. Note that the spread in accelerator-hours, 35.9×, is wider than the spread in energy: a fleet-sizing decision and an electricity bill are not the same comparison.

On that basis C++ serves the same volume at 0.43× the baseline energy and Java at 9.63×. The inference blocks these multipliers come from are sub-second in most stacks, which is the regime Section 6.5 identifies as least reliable for any instrument; they are counter-bracketed and so do not carry the estimator's padding, but they are the shortest measurements in the campaign and the ratios inherit that.

**Takeaway.** Ecosystem-level energy differences, treated purely as relative multipliers under a uniform protocol, translate into large operational deltas at hyperscale. Two caveats bound that statement, and both come from this study rather than from convention. The multipliers were measured on a single-accelerator desktop at $32 \times 32$ input resolution, and on inference blocks that are sub-second in the fastest stacks; nothing in this study establishes how they transfer to production model sizes, and we can bound neither the direction of that error nor its size. The one contrast we did measure shows why: at $224 \times 224$ (Section 6.6) the inference multipliers narrow within the LibTorch lineage and move in opposite directions across all 7 stacks on the two architectures. And the whole calculation is a scaling of *relative* differences: had we reported the modelled component of a software estimator's output as if it were measured, the same arithmetic would have produced a confident number from a quantity that depends on how much memory the host happens to have.

# I The catalogue of defect classes, in full

Section 8 summarises this catalogue.

Building this study meant first auditing the campaign behind an earlier version of it, the audited campaign, and then repairing every stack against the specification. Table I.1 collects what that turned up. We report the defects as *classes* rather than as a list of bugs in one project, because each is a plausible mistake in any study of this shape, several are language or tool *defaults* rather than oversights, and most are invisible in the resulting measurements.

The last column is the one that matters. It asks whether a reader given only the released data — the tables, the CSVs, the plots — could detect the problem.

One row of that table deserves to be made concrete, because it is the class most likely to affect any study of this shape and the least likely to be noticed. Table I.5 reports the training protocol *as implemented*, read from the source of the audited campaign, whose manuscript stated a single common learning rate.

Two stacks train at ten times the stated learning rate. Loader parallelism ranges from none to every hardware thread on the machine, and across the campaign it correlates with epoch duration at $\rho = -0.73$ ($p = 0.041$) over a $11.6\times$ spread in duration and $9.9\times$ in energy — which is to say that a column no one would think to report tracks the published ranking better than the programming language does. One stack normalises its inputs when no other does, so it is not even computing the same function. None of this is visible in an energy figure, and all of it is verifiable in about a page of code. The check we wrote to catch the learning-rate column passed on it, for the reason Section B.3 gives: it asked whether some file matched, where the specification says every file must.

Three observations follow.

**Only three of these are visible from the released data.** One changes no conclusion, since a constant factor leaves every ranking intact. The second announces itself as an epoch count that does not match the protocol — visible, but only to a reader who thinks to check it against the specification, which is the whole reason for writing one down. The third is a test set silently shortened by sixteen images, and it

announces itself only to a reader who multiplies a reported accuracy by the number of images it is supposed to be over. Everything else that changes a conclusion requires reading the code, or a second instrument, or a quality metric the study has to have thought to record.

**Several are defaults rather than oversights.** The linker's `--as-needed` behaviour, an estimator's fallback power constants, a framework resolving CUDA wheels that do not match its build, a resize function that returns `uint8` because the images it was written for were `uint8`. None is a lapse in care. A study can be executed carefully and land on all of them.

**Some of them are ours.** 12 of the 34 entries were introduced while building or repairing this study. Most were caught by a mechanism rather than by attention: a conformance check, the accuracy metric, the second instrument, a parity fingerprint, or the campaign driver refusing to start. Two were not. One was found by reading an accelerator that was drawing idle power, after a day in which five plausible hypotheses were each measured and refuted; the other by the campaign itself failing on its first batch with every check green.

One of them deserves singling out, because it is not a mistake in the usual sense. Moving the input transform into the loader is required by S3. Switching evaluation to batched inference is required by S3 as well. Each change is correct, and each was reviewed. Together they left a private copy of the transform on the training path and removed it from the evaluation path, and the result trained a network on black images for thirty epochs while reporting a plausible falling loss. No amount of care applied to either change in isolation would have caught it; what caught it was a metric neither change was about. We report them because the alternative — a paper claiming its authors made no mistakes — would undercut the argument it is making. The argument is that enforcement beats discipline, and the evidence for it is that our own discipline was not sufficient either. The qualification the latest campaign added is where the enforcement has to reach. An unenforced clause is a clause that will drift; a clause enforced against source rather than against the built binary is barely enforced at all; and a clause enforced against both still cannot tell you that the code takes a step. One unenforced clause survives in the campaigns reported here: C++, and Rust in the $32 \times 32$ campaign, are handed the expected parameter count and never check it, and the conformance check for that clause does not look at either. We found it after the campaign and the saturation cell had run and report it rather than patch a harness that has already produced the data (Section 4.1).

**Table I.1**: Defect classes of the instrument, with the measured effect of each; Tables I.2–I.4 continue the catalogue with the implementation, placement and analysis classes. "Visible" asks whether a reader given only the released measurements could detect the problem. Entries marked ∗ are ours: defects we introduced in the harness we built for this study. The unmarked entries were either found in the campaign this study audits or are framework and library defaults we had to override. What caught the marked ones differs by row — a conformance check, the accuracy metric, the second instrument, a parity probe, a power reading, and in one case only the campaign itself failing at its first batch — and the mixture is the point: no single mechanism found them all.

| Class | Measured effect | Visible |
|---|---|---|
| *Instrument* | | |
| Unit mislabelling | Energy reported in kW h under a J label; a factor of $3.6 \times 10^6$ | yes |
| Reported duration is not the accumulation window | 75 % of blocks padded by seconds of tracker lifetime in 3 discrete modes; power understated 13.0× on the shortest blocks | no |
| Modelled term inside a measured total | RAM energy from installed capacity in machine mode, 7.6 % of the reported total; scales with the host, not the workload | no |
| Mixed instrument versions | Versions 2.8.4 and 3.0.4 in one campaign: 231 W against 102 W of modelled host power for the same machine | no |
| Mixed tracking mode | 1 of 8 stacks tracked per-process rather than machine-wide; its RAM term is 0.026 % of reported energy against 25 % to 55 % for the rest | no |
| Interpreter resolved from `PATH` | The instrument's version becomes a property of the shell; 5 of 8 stacks report a single constant CPU power, the signature of a modelled fallback | no |
| Unsynchronised tracker | Inference recorded as 0 J; training understated 24 % | no |
| Host not idle | Whole-machine tracking charges unrelated downloads to the workload | no |
| Two campaigns on one accelerator* | An orphaned driver restarted and wrote the same run directories while sharing the GPU: runs of 59, 60 and 43 epochs where the specification says 30, each measured against the other's contention | yes |
| Device work outlives the window* | A framework that dispatches asynchronously had its inference block closed before the device finished: a third of the work outside the window, and energy reported at 0.70× the figure the same block gives when the stack is forced to synchronise | no |
| A wrap guard with the wrong units* | The RAPL multi-wrap guard divided a counter range in µJ by a power in W, which put its threshold six orders of magnitude out — 59 years rather than 31 minutes — and made it inert for every realisable block; it was also consulted only when a delta came back negative | no |
| Manifests without machine state* | No power limit, clock, persistence mode, governor, boost, driver version or precision setting recorded for any run, so nothing in the record establishes that two campaigns ran on the same machine in the same state | no |

**Table I.2**: Defect classes, continued: what the stacks compute. Columns and marks as in Table I.1.

| Class | Measured effect | Visible |
|---|---|---|
| *Implementation* | | |
| Divergent hyperparameters | 2 learning rates across 8 stacks | no |
| Divergent loader parallelism | 0 to 96 threads; $\rho = -0.73$ against epoch duration ($p = 0.041$) over a 11.6× spread | no |
| Divergent input shape | One stack at 0.26× another's `conv1` input volume; 0.07× the energy | no |
| Divergent evaluation batching | Batch 1 against 128; 1.5–1.95× inference energy | no |
| Evaluation in training mode | Test accuracy 21 % against 86 %; a different computation measured | no[†] |
| Resize discards a dtype conversion* | Image scaled to $[0, 1]$, then resized by a function returning `uint8`; the scaling is silently thrown away | no[‡] |
| Two correct changes that are wrong together* | Moving the transform into the loader, and batching evaluation, each right alone; together they left a private copy of the transform on the training path only, which trained the network on black images for thirty epochs | no[†] |
| Channel order assumed, not checked* | `[C,H,W]` treated as `[H,W,C]`: the width cropped to three pixels and the result transposed | no[‡] |
| Ambiguous dataset layout | A class name containing `/` yields three different datasets | no |

[†] invisible because no stack wrote its accuracy to disk; visible the moment one does.
[‡] invisible in energy data; here it happened to abort the run, but the channel-order defect would have trained quietly on a three-pixel sliver had the dtype defect not stopped it first.

**Table I.3**: Defect classes, continued: what the stacks compute, second part. Columns and marks as in Table I.1.

| Class | Measured effect | Visible |
|---|---|---|
| *Implementation, continued* | | |
| A precision flag set on one axis in one stack* | One stack of 7 pinned true-fp32 convolutions, one was handed a mixed-precision policy, one was handed nothing and four took the driver's default; 3.62× the accelerator energy per step separates the two branches, and no column of the metric files distinguishes them | no |
| Four networks under one name* | VGG-16 built independently in four frameworks spanned an order of magnitude in parameters, under a specification clause stating that counts were checked against the exported module and a checker in which no such check existed | no |
| Framework default initialisers diverge | Two frameworks' default distributions were 4.6× and 3.3× wider than the reference at the stem of the same architecture: the same graph, built from a different distribution, which a parameter count cannot see | no† |
| `momentum` means opposite things in three libraries | Batch-normalisation momentum is a retention factor in one framework and an update factor in another, and a third defaults to 0.99 with $\epsilon = 10^{-3}$: nine points of test accuracy on one stack, in the denominator of the efficiency metric | no† |
| Seeds that no stack used* | 3 stacks of 7 were handed a distinct seed per repetition and ignored it — one hardcoded, one never forwarded, one rebuilt inside the epoch loop — while the conformance check reported five distinct seeds, having asked the campaign planner rather than the stacks | no |
| `drop_remainder` silently changes the test set | Two stacks took fewer gradient steps per epoch and were scored on 9 984 of 10 000 test images, always the same 16; detectable in the released accuracies, since test_acc × $N/100$ is an integer only for the true $N$ | yes |
| A resampling filter's default | 4 stacks of 7 decoded different pixels on the one dataset that has to be downsampled; means agreeing while variances did not, over a third of the campaign | no |
| A normalisation applied twice* | One model in one stack divided by 255 an array its loader already returned in $[0, 1]$, so it trained on $[0, 1/255]$ under a clause fixing the pipeline for every stack; the batch normalisation after the first convolution absorbed it and the accuracies never showed it | no |

† invisible because no stack wrote its accuracy to disk; visible the moment one does.

**Table I.4**: Defect classes, continued: where the stacks compute, and how the measurements were aggregated. Columns and marks as in Table I.1.

| Class | Measured effect | Visible |
|---|---|---|
| *Placement* | | |
| Silent CPU fallback | Whole workload on the CPU after a CUDA wheel mismatch; one warning line | no |
| Linker drops the CUDA dependency | Same source, same LibTorch: `Cpu` against `Cuda(0)` | no |
| *Analysis* | | |
| Collapsed runs counted as measurements | 12 first-campaign runs at chance accuracy averaged in with runs that trained, and the rate read as an ecosystem property rather than as a framework initialiser default | no[†] |
| Partial runs averaged as measurements[*] | A crashed run's fragment, 16 J against a neighbour's 300 kJ, produced ecosystem spreads of $2 \times 10^4$ | no |
| Epochs treated as replications | Effective sample size 1 per configuration; intervals far too narrow | no |

[†] invisible because no stack wrote its accuracy to disk; visible the moment one does.

**Table I.5**: Training protocol as implemented in the audited campaign, the one behind an earlier version of this study, read from source rather than from the manuscript. The manuscript stated one learning rate and one input scaling; the code contained two of each. Divergences in bold. Specification S2 and S3 now fix every column; the conformance checks verify the learning rate against the source for every stack and the input scaling for one stack only, and the remaining columns are asserted by the specification and were checked by reading.

| Ecosystem | Optimiser | LR | Loader mechanism | Threads | Input scaling |
|---|---|---|---|---|---|
| Python/PyTorch | Adam | 1e-4 | DataLoader workers | 2 | $[0, 1]$ |
| Python/TensorFlow | Adam | **1e-3** | Keras generator | 1 | $[0, 1]$ |
| Python/JAX | Adam | 1e-4 | Keras generator | 1 | $[0, 1]$ |
| C++/LibTorch | Adam | 1e-4 | DataLoader workers | 2 | $[0, 1]$ |
| Java/DL4J | Adam | 1e-4 | AsyncDataSet-Iterator | 2 | $[0, 1]$ |
| R/torch | Adam | 1e-4 | dataloader workers | **0** | $[0, 1]$ |
| Rust/tch | Adam | **1e-3** | rayon, all cores | **96** | **mean/std** |
| MATLAB/DLT | adam | 1e-4 | augmentedImage-Datastore | 1 | $[0, 1]$ |

# J Threats to validity: the extended discussion

Section 9 condenses the threats to validity. This section gives the discussion in full, threat by threat, with the evidence behind each.

As with any empirical study, our work is subject to threats to validity. We group them conventionally, and note where a threat that would ordinarily be speculative is one we can quantify from the data.

**The reported campaign replaces one of our own, and we say what changed.** The superseded campaign, of exactly this design, ran to completion and produced a coherent set of numbers. Auditing it against its own source afterwards found that three of its headline claims were confounded: one numerical-precision policy was in force for some stacks and not for others, the 7 stacks were training four different networks under the single name VGG-16, and the weight initialiser differed across stacks. We aligned the stacks — one precision policy, one canonical module per architecture, one initialiser — and re-executed all 210 runs. The same pass closed the asynchronous stack's measurement window, restored the test set two stacks had been scored on, brought three incompatible batch-normalisation conventions together, and threaded the seeds that three stacks had been handed and did not use. Every ecosystem comparison in this paper is measured after that alignment. Where the paper reports the campaign behind an earlier version of this study — the protocol read from its source in Table I.5, and the defects it contributed to the catalogue — it is named the audited campaign.

What the re-execution leaves standing is the apparatus, and it leaves the findings rather than the figures. The instrument comparison of Section 6.5, the padding and window-floor results and the boundary argument are statements about the tooling rather than about any one campaign's numbers: every quantity in them is re-derived from this campaign like everything else here, and several of them moved, but what they establish did not. The same holds for the source-level defects in the catalogue of Section 8: they were found by reading code, and re-running the code does not unfind them. The distinction is the one this paper asks its readers to make everywhere else — a result about what the stacks did needs the stacks to have been doing the same thing, and a result about the instrument does not.

**Internal Validity.** Whole-machine energy tracking means any concurrent work contaminates a measurement. The host was otherwise idle for the duration of the campaign; the blocks measured during development while a package download was running were discarded rather than corrected, and the protocol amended to forbid it. We report this because it is an easy mistake to make and an invisible one to inherit.

Toolchain heterogeneity remains. CUDA versions across the 7 stacks range from 11.6 (Deeplearning4j, whose final release predates CUDA 12) to 12.8 (the pinned LibTorch shared by the control group). This is not fully controlled and cannot be: an ecosystem is partly defined by the toolchain it supports. What we have changed is that it is now *bounded*. The 3 stacks of the control group are pinned to one build, and the precision policy is set campaign-wide rather than per stack (Section 6.2.2), so Table 4 separates the spread that survives a common model and a common backend from the spread that does not. It does not say what the surviving spread is. Pinning a build is

not by itself an isolation of any one cause, and a policy written into a specification is not the same thing as a policy every stack receives.

Implementation drift between stacks was the dominant internal threat in studies of this shape, and it is the one this design attacks directly. Under specification S1 the 3 stacks of the control group load the same exported module rather than independent reimplementations, five stacks assert their parameter count at startup — C++ and the campaign's Rust binaries receive it and do not check it, which we found after the campaign and the saturation cell had run — and all 7 agree shape for shape on both architectures; S2 and S3 fix the optimisation and the data pipeline; and 110 automated checks verify the source-level and artefact-level clauses against the source, the built binaries and the campaign's own metric files, with shape-for-shape parity checked separately by `scripts/verify_architecture_parity.py`. 12 defects we introduced while building or repairing this study are catalogued in Section 8; most were caught by a mechanism — a check, the accuracy metric, the second instrument, or the driver refusing to start — and two were caught by neither a check nor a review, which is why we do not claim the specification is complete. We claim only that a divergence outside it now has to survive an executable check, and that the campaign itself is the last check in the sequence.

**We applied our own fallback diagnostic to this campaign.** The defect catalogue names the lowest mean accelerator power among the stacks as the signature of a stack that silently ran on the host, and in this campaign one stack trips it: R draws 127 W at the accelerator against up to 258 W for the others, on a 350 W card. A reader applying our criterion would suspect the stack that produces our largest reported ratios.

The CPU term clears it. If R were running the workload on the host its package power would rise, and it does not: 53 W, inside the 50–68 W band every other stack occupies, and the accelerator assertion of S4 refuses a CPU device at startup. R is not running the workload on the host in place of the accelerator; it uses the accelerator at a low duty cycle, and its epochs are the longest in the campaign. We state this because a diagnostic published in a paper should be run against that paper's own data, and because the exoneration is a measurement rather than an assurance.

We can say what that duty cycle is. The accelerator was sampled at 1 Hz alongside the campaign, and across the 157 of 210 runs the sampler covered, R's reads 4.5–5.0 % utilisation on ResNet-18 and 12.1–13.4 % on VGG-16, with 1 271–3 001 MiB of device memory held by the R interpreter: the card is attached, resident and computing, and idle most of the time. These figures are taken over each run's whole wall clock, start-up and evaluation included; the utilisation contrast of Section 6.6 uses the training blocks alone on both of its sides, a different window whose figures are not comparable with these. R runs the two loader workers the specification mandates, the same two every stack gets. The other signature is in the spread rather than the level — across its runs R's accelerator power stays between 126 W and 141 W whichever network it is training, where every other stack's moves several times as far with the network.

Accelerator placement was verified rather than assumed. Every harness asserts the device at startup and refuses to fall back to the CPU. This is not hypothetical: linking a Rust binary against a CUDA LibTorch produces, by default, a binary that runs

entirely on the CPU, because `--as-needed` drops the `torch_cuda` dependency that no Rust symbol references. Nothing in the energy data distinguishes that case from a correct one except its magnitude, which is why the check that catches it reads the binary rather than the energy: `readelf -d` on every Rust binary we build, refusing any whose dynamic section names no CUDA library.

**Construct Validity.** Our construct is energy at a stated boundary, not "sustainability". We report accelerator-board plus CPU-package energy read from hardware counters, which excludes the power supply, fans, drives, chipset — and host DRAM, for which this platform exposes no counter. We had no wall meter and therefore do not report a whole-system figure, and a board-and-package number must not be compared with a wall-meter number from another study.

The counters are our reference and they are not exempt from the scrutiny we apply to the estimator. RAPL's package energy is itself a modelled quantity on some parts and its update interval is of order a millisecond (Khan et al. 2018); NVML's accumulated-energy register rests on board telemetry averaged internally, so its temporal resolution is not unlimited either. That bounds what the sub-second comparison of Section 6.5 can establish: for the shortest blocks in this campaign, both instruments are near the edge of what they resolve, and the disagreement we report there should be read as a joint property of the pair rather than as a fault of one. It does not bear on the reported-duration finding, which concerns a field rather than a measurement.

Two things about the counters changed for this campaign and both belong here. The first is a defect of ours. The guard that detects the RAPL energy register wrapping compared a reading in microjoules with a figure in watts, so the interval it computed before a second wrap became possible was decades rather than the tens of minutes it intended, and the guard could not fire; it was also consulted only when a delta came back negative, which is not what two wraps inside one block produce. No reported number moves — the longest block in this campaign is 95.7 s, far short of one wrap — but the guard had been inert for every block we have ever measured, and a guard that cannot fire reads exactly like a guard that never had to.

The second is that a measurement is only as reproducible as the machine state it was taken in, and ours recorded none of it: no power limit, no clock policy, no persistence mode, no ECC state, no governor, no boost setting, no driver version, not even a timestamp. A replicator could see every version of every library and nothing at all about the card. Every run of this campaign records that state in its manifest, written by both output paths so that the four stacks which never import our bridge are described as fully as the three that do. Provenance is the part still short: the harness now writes the source revision and a hash of the measurement path into each manifest, and the runs analysed here predate that, so which code produced them is argued from commit dates against run timestamps rather than read off the artefact.

That recording is owed to an observation we cannot explain. Two readers of the audited campaign's data noticed independently a bimodal accelerator power state, and we can say nothing more about it than that: none of that campaign's manifests records any machine state, so the conditions it was observed under cannot be recovered from the artefact, and we offer no mechanism. What is different now is that every run of this campaign carries the state a replicator would need — persistence mode,

power limit, clocks, governor, boost and driver version — so a recurrence would be attributable rather than merely reported.

A second construct limitation belongs here, because it bounds what the attribution in Section 7 can claim. Every figure we report is un-baselined: the counters accumulate whatever the silicon draws, the machine's static consumption included. Un-baselined is not whole-machine, and the two are worth keeping apart — the boundary is the accelerator board plus the CPU package, and what a wall meter would add to it is quantified above. We measured that directly rather than arguing about it. On an idle host at the same boundary this machine draws 64 W — 25 W at the accelerator and 39 W at the CPU package — which is 23 % of the mean power of a measured block.

The two halves of the boundary are affected very differently, and this is the part that matters for interpretation. The accelerator's idle draw is 11 % of its campaign mean, so the GPU term is mostly workload. The CPU package's is 68 %: of the 58 W mean package power, the great majority is the machine being on. That term accordingly carries almost no workload signal — its coefficient of variation across the whole campaign is 12 % against 35 % for the accelerator — while contributing 21 % of reported energy. That is one reason we do not apportion the ecosystem spread between host-side and accelerator work: this boundary carries no measured host-side energy term to apportion, and neither accelerator idle time, the control group of Table 4, nor the contrast cell of Section 6.6 substitutes for one. The cell is consistent with a host-side share in R's cost; it does not measure that share. A study wanting to measure host-side work directly would need DRAM and platform counters this machine does not expose. We report un-baselined energy because that is what the counters give and what an operator pays for, and we state the baseline so a reader can subtract it.

The more interesting construct threat is one we can measure. A software estimator's output is a mixture of measured and modelled terms, and the mixture is not stated in the output. Over 12 600 blocks, CodeCarbon agrees with the counters to 0.3 % on the terms both measure, while its total exceeds them by 1.087× — the difference being a RAM term derived from installed capacity, 7.6 % of the reported total here and necessarily larger on a large-memory host. A figure of that construction corresponds to no physical boundary, and its value depends on the host's memory configuration rather than on the workload.

The estimator's reported duration is a second construct threat, and the sharper of the two: it carries seconds of tracker lifetime in 3 discrete modes rather than being the accumulation window, so any power or energy–time quantity derived from it is biased — most heavily on the shortest phases, though block length predicts which mode a block lands in without determining it. We use counter-bracketed durations throughout for this reason.

**External Validity.** We studied two convolutional architectures and three image classification datasets at $32 \times 32$ resolution, and one contrast cell at $224 \times 224$; results may not generalise to other model families, to transformers, or to other resolutions and batch sizes, where the balance between accelerator work and host-side work is different. The contrast cell shows that this balance moves the stacks by different amounts and in different directions, so the campaign's multipliers are not constants to be carried to another workload.

The campaign's per-sample compute is small. Over the training blocks 1 of the 7 stacks holds the accelerator at or above 85 % on ResNet-18, against 5 on VGG-16 (Section 6.6), so what the campaign compares is whole pipelines rather than saturated kernels. Section 6.6 reports one contrast at higher per-sample compute, and that contrast has limits of its own. It changes resolution, batch size and dataset together, so it cannot attribute a change to any one of them. It is one point, with batch 32 forced by VGG-16's memory at $224 \times 224$. It ran at 2 loader workers only, and R stays short of saturation there for a reason the cell does not isolate. And the cuDNN reference point of Section 4.1 was not repeated at $224 \times 224$, so the part of Java's distance owed to its missing convolution backend is not separated there.

The platform is a single-accelerator desktop rather than a datacentre node, which makes the host share of total energy smaller than it would be on a multi-socket server. And MATLAB, present in the audited campaign, is out of scope here because it cannot be brought into conformance with the specification; readers should not read its absence as a result about MATLAB.

**Conclusion Validity.** Many analyses of studies of this shape — including an earlier one of our own — treat the epochs of a single run as repeated measurements. They are not independent: epochs within a run share an initialisation, a JIT outcome, an allocator state and a thermal trajectory, so the effective sample size per configuration is one, confidence intervals are far too narrow and significance is inflated. Every interval and test reported here is computed on 5 independent runs per configuration, and 206 of the 210 runs were interleaved rather than executed consecutively, so that machine-state drift is spread across conditions instead of aliasing onto one ecosystem. The exception is stated at the end of this subsection.

Five repetitions is a small sample for a rank test, and we report what that costs rather than working around it. A Kruskal–Wallis test rejects equality of ecosystems in all 12 test groups ($p \leq 1.2 \times 10^{-5}$, $\varepsilon^2 \geq 0.95$), and 99 % of the 252 pairwise comparisons carry a large Cliff's delta. That effect size is saturated too, and we say so rather than leaning on it: with a between-run coefficient of variation under half a per cent, five-run groups almost never overlap, so $\delta$ reaches $\pm 1$ for a $1.1\times$ difference exactly as it does for a $38.7\times$ one. It records that the groups separate, not by how much. Yet 0 of those 252 pairs is significant after Holm correction — and cannot be, at any effect size. With five runs against five, the smallest attainable two-sided exact Mann–Whitney $p$ is 0.0079, and Holm over the 21 comparisons within a group multiplies it to 0.167. The design has the power to establish that the ecosystems differ and not the power to order any particular pair; we therefore rest the ordering claims on effect sizes and intervals, and make no pairwise significance claim.

Between-run variability is low — a median coefficient of variation of 0.49 % for training energy — which makes the design well powered for the effect sizes we report. Low variance is a statement about the measurement rather than a guarantee that the measurement is of the right thing.

Where the sample is genuinely too small we say so rather than reach, and the collapse rates of Section D.3 are where we did not. In the superseded campaign those rates differed across ecosystems; we tested that table with a $\chi^2$ test and with a permutation test and reported both, because they disagreed. The disagreement was a property of

the two statistics rather than evidence about the data. $\chi^2$ wants five expected observations per cell and that table gave it 1.7; our permutation statistic was the *range* of per-ecosystem rates, which sees only the extremes and cannot tell 7 ecosystems at 0, 0, 0, 10, 20, 40 and 50 per cent from two at 0 and 50 with five sitting at the mean. What we got wrong was running either test at all. Neither was the right test for that table, and the table was measuring the initialiser rather than the ecosystem.

The table warranted an exact test. Freeman–Halton, the $r \times c$ generalisation of Fisher's exact test, conditions on both margins and sums the probability of every table no more likely than the one observed: a few thousand terms, no approximation and no seed. On the superseded campaign's table it returns $p = 0.0040$. On the reported campaign's it returns $p = 1.0000$, because no ecosystem collapsed a run and there is nothing left to be inhomogeneous — with zero events in every cell the question is not which ecosystems differ but how tightly the rate is bounded, which is the 4.2 % of Section D.3 and not a significance test at all. We keep $\chi^2$ in the analysis only as the contrast we argue against, guarded so that a zero expected frequency reports no value rather than stopping the pipeline.

Finally, 4 runs — 3 configurations, all of them JAX — were executed in a separate window 2 days after the rest of the campaign. They cover VGG-16/CIFAR-100, VGG-16/Fashion-MNIST, VGG-16/Tiny ImageNet. They are therefore not interleaved with the other ecosystems, and machine-state drift between the two windows is not controlled for them by the design.

They are replays, and what they replace is worth stating. The original runs failed early in the campaign, on their first training batch: the shared classifier head carries a dropout layer, and our JAX implementation called the network without the random-number stream that layer requires. The failure came after the tracker had opened a block and recorded a parameter count, and the count was correct, because dropout has no parameters. Every check in the suite had passed on that code. We fixed it before the remaining JAX VGG-16 runs executed, so every JAX VGG-16 run in the campaign comes from one source revision, and the runs that predated the fix were deleted and replayed rather than repaired in place.

We measured it rather than reasoning about it, because the reassurance the argument wants — that the between-run variability is small — bounds variability *within* a window and says nothing about drift across days. We re-executed one already-completed configuration (PyTorch, ResNet-18 on Fashion-MNIST) in a third window, on the same machine under the same idle conditions and with the same seeds, and compared it run for run with its originals. The comparison is refused outright unless both windows record the same precision policy, and unless the calibration postdates the campaign it calibrates: a calibration produced by an earlier harness measures that harness, not the machine, and the first one we had for this campaign was exactly that.

Training energy in the later window is 1.2 % lower, which is 1.6 standard deviations of the within-window spread — the same order as the noise the design already carries, and small against the between-ecosystem differences it could confound. Inference is 3.2 % lower, 4.0 standard deviations against a within-window coefficient of variation of 0.9 %. That is several times the within-window spread and we do not call it small:

in absolute terms it is 5.6 J per block, on the sub-second inference blocks that are the noisiest measurement in the campaign.

We therefore treat the late-window training figures as comparable and the late-window inference figures as carrying a between-window uncertainty of the order of 3.2 %, which is small against the differences between ecosystems reported in Section 6.3 but is not noise. It is a bound obtained from one configuration, in a different ecosystem and on a different architecture from the 3 it is applied to.

## References


Alizadeh N, Castor F (2024) Green AI: A preliminary empirical study on energy consumption in DL models across different runtime infrastructures. In: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering — Software Engineering for AI (CAIN), pp 134–139, https://doi.org/10.1145/3644815.3644967

Chattaraj R, Chimalakonda S, Sharma VS, et al (2025) Who wins the race? (R vs Python) — an exploratory study on energy consumption of machine learning algorithms. arXiv preprint arXiv:2508.17344

Courty B, Schmidt V, Luccioni S, et al (2026) `mlco2/codecarbon`: v3.3.0. https://doi.org/10.5281/zenodo.4658424, URL https://github.com/mlco2/codecarbon

Cruz L, Fernandes JP, Kirkeby MH, et al (2025) Greening AI-enabled systems with software engineering: A research agenda for environmentally sustainable AI practices. ACM SIGSOFT Software Engineering Notes https://doi.org/10.1145/3743095.3743099, arXiv:2506.01774

Feldt R, Magazinius A (2010) Validity threats in empirical software engineering research – an initial survey. In: Proceedings of the 22nd International Conference on Software Engineering & Knowledge Engineering (SEKE), pp 374–379

Georgiou S, Kechagia M, Sharma T, et al (2022) Green AI: Do deep learning frameworks have different costs? In: Proceedings of the 44th International Conference on Software Engineering (ICSE), pp 1082–1094, https://doi.org/10.1145/3510003.3510221

Gordillo A, Calero C, Moraga MÁ, et al (2024) Programming languages ranking based on energy measurements. Software Quality Journal 32(4):1539–1580. https://doi.org/10.1007/s11219-024-09690-4

Jay M, Ostapenco V, Lefèvre L, et al (2023) An experimental comparison of software-based power meters: Focus on CPU and GPU. In: Proceedings of the 23rd IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pp 106–118, https://doi.org/10.1109/CCGrid57682.2023.00020

Khan KN, Hirki M, Niemi T, et al (2018) RAPL in action: Experiences in using RAPL for power measurements. ACM Transactions on Modeling and Performance

Evaluation of Computing Systems 3(2):9:1–9:26. https://doi.org/10.1145/3177754

Li D, Chen X, Becchi M, et al (2016) Evaluating the energy efficiency of deep convolutional neural networks on CPUs and GPUs. In: Proceedings of the 2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom), pp 477–484, https://doi.org/10.1109/BDCloud-SocialCom-SustainCom.2016.76

Li H, Rajbahadur GK, Bezemer CP (2024) Studying the impact of TensorFlow and PyTorch bindings on machine learning software quality. ACM Transactions on Software Engineering and Methodology 34(1):1–31. https://doi.org/10.1145/3678168

Marini N, Pampaloni L, Di Martino F, et al (2025) Green AI: Which programming language consumes the most? In: 2025 IEEE/ACM International Workshop on Green and Sustainable Software (GREENS), arXiv:2501.14776

Pereira R, Couto M, Ribeiro F, et al (2021) Ranking programming languages by energy efficiency. Science of Computer Programming 205:102609. https://doi.org/10.1016/j.scico.2021.102609

Sjøberg DIK, Bergersen GR (2023) Construct validity in software engineering. IEEE Transactions on Software Engineering 49(3):1374–1396. https://doi.org/10.1109/TSE.2022.3176725

Verdecchia R, Sallou J, Cruz L (2023) A systematic review of Green AI. WIREs Data Mining and Knowledge Discovery 13(4):e1507. https://doi.org/10.1002/widm.1507